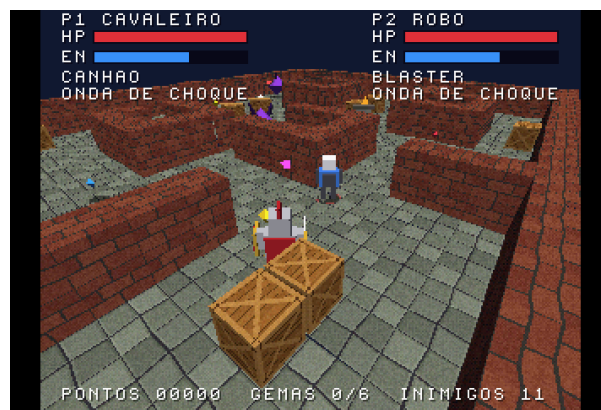


PS1 Arena

Manual do desenvolvedor

Como alterar o código para criar fases, personagens, armas, poderes, inimigos, som, saves e o modo online



Projeto	PS1 Arena v3 — jogo 3D de PlayStation 1 em C
SDK	PSn00bSDK 0.24 (commit fixo no Dockerfile)
Compilador	GCC mipsel-linux-gnu 12.4 · CMake 3.28 · Ninja
Ambiente	Windows + WSL2 Ubuntu + Docker (também Linux/macOS)
Público	programadores que vão evoluir o jogo
Data	outubro de 2026

Sumário

1. Como usar este manual	5
1.1 Organização	5
1.2 As receitas foram compiladas e testadas	5
1.3 Convenções	5
2. Ambiente e ciclo de trabalho	6
2.1 O script ./dev	6
2.2 O que entra no build sozinho	6
2.3 Ciclo recomendado	6
3. Arquitetura do jogo	7
3.1 Mapa dos módulos	7
3.2 O laço principal e o passo fixo	8
3.3 Máquina de estados	9
3.4 Um quadro de desenho	9
3.5 Estado global e pools	10
3.6 Regras de ouro do projeto	10
4. O PlayStation 1 para quem vem de back-end	11
4.1 Ponto fixo	11
4.2 Ângulos e trigonometria	11
4.3 Eixos e unidades	12
4.4 GTE, Ordering Table e zbias	12
4.5 VRAM e texturas	13
4.6 Orçamento de memória	13
5. Referência de tipos	14
5.1 Definições (tabelas de data.c)	14
5.2 Entidades (estado em g)	15
5.3 Desenho e modelos	16
6. Referência de funções	17
7. Receitas: visão geral e ajustes rápidos	19
7.1 Ajustes por constante (config.h)	19
7.2 Ajustes por tabela (data.c)	20
8. Personagens	21
8.1 Do Blender para o jogo	21
8.2 Adicionando o personagem	22
8.3 Skins	22
8.4 R7 — Animação por poses	22
9. Armas e projéteis	24

9.1 Como um tiro acontece	24
9.2 Arma nova só com a tabela	24
9.3 R5 — Arma explosiva (lança-granadas)	25
9.4 Outras ideias	26
10. Poderes	27
10.1 Poder simples	27
10.2 R6 — Congelar inimigos	27
11. Inimigos	29
11.1 Como a IA funciona	29
11.2 Inimigo novo só com a tabela	29
11.3 R1 — Inimigo que atira	30
11.4 R2 — Chefe com barra de vida	32
11.5 Mais comportamentos	32
12. Itens e objetos de cenário	33
12.1 Como os itens funcionam	33
12.2 R4 — Item de velocidade	33
12.3 Objetos de cenário (props)	34
13. Fases	35
13.1 O mapa em texto	35
13.2 R3 — Várias fases e progressão	35
13.3 Indo além	38
14. HUD, telas e câmera	39
14.1 Desenho 2D	39
14.2 R12 — Um estado novo: introdução da fase	39
14.3 Câmera	40
15. Colisão	41
15.1 Uma entidade nova que colide	41
16. Som, CD e memory card	42
16.1 R8 — Efeitos sonoros (SPU)	42
16.2 R9 — Ler arquivos do CD	44
16.3 R10 — Recorde no memory card	45
17. Modo online	46
17.1 As opções	46
17.2 Streaming com Parsec (sem mudar o código)	46
17.3 Modo link: a ideia	47
17.4 Requisitos de determinismo	48
17.5 O protocolo	49
17.6 R11 — Implementação	49

17.7 Testando com dois PCSX-Redux	54
17.8 Pela internet e em consoles reais	54
17.9 Depurando dessincronias	54
17.10 Limitações e próximos passos	55
18. Desempenho e memória	56
18.1 Orçamentos práticos	56
18.2 Técnicas	56
19. Depuração	57
19.1 Ferramentas	57
19.2 Falhas comuns	57
20. Fluxo de trabalho, versões e entrega	58
20.1 Git	58
20.2 Ao terminar uma mudança	58
20.3 Lista de verificação de release	58
20.4 Gravar em CD	58
20.5 Trabalhando com um agente de IA	58
21. Solução de problemas	59
Apêndice A — Caracteres do mapa	60
Apêndice B — Constantes de config.h	61
Apêndice C — Botões (psxpad.h)	62
Apêndice D — Glossário	63
Apêndice E — Roteiro de exercícios	64

1. Como usar este manual

Este manual é para quem vai **pegar o código do PS1 Arena e transformá-lo**: novas fases, personagens, armas, poderes, inimigos, telas, som, saves e jogo entre dois consoles. Ele complementa o [README.md](#) (referência técnica e ambiente) e o [docs/GUIA.md](#) (receitas curtas). Aqui o foco é **o código**: onde cada coisa mora, por que foi feita assim e o passo a passo para mudar.

1.1 Organização

Parte	Capítulos	Para quê
Fundamentos	2 a 6	ambiente, arquitetura, conceitos do PS1, referência de tipos e funções
Receitas	7 a 16	modificações passo a passo, com código completo
Modo online	17	Parsec hoje; modo link pela serial (lockstep) com implementação
Engenharia	18 a 21	desempenho, depuração, fluxo de trabalho, solução de problemas
Apêndices	A a E	caracteres do mapa, constantes, botões, glossário, exercícios

1.2 As receitas foram compiladas e testadas

Todo código das receitas (R1 a R12) foi **aplicado numa cópia do projeto e compilado** com o mesmo Docker do `./dev build`, sem erros nem avisos. As receitas de jogabilidade (inimigo atirador, chefe, segunda fase, item de velocidade, tela de introdução) também foram **executadas num emulador** — as capturas de tela ao longo do texto vêm desse teste. As exceções estão marcadas:

- **R8 (som)**, **R9 (CD)** e **R10 (memory card)** compilam e usam a API documentada do PSn00bSDK, mas não foram ouvidas/lidas num console; trate-as como ponto de partida.
- **R11 (modo link)** compila com `LINK_MODE` ligado, mas não foi testada entre dois emuladores. O capítulo 17 explica como testar.

1.3 Convenções

Notação	Significado
<code>src/data.c</code>	arquivo, relativo à raiz do projeto
<code>weapon_fire()</code>	função; <code>weapon_defs[]</code> é uma tabela (vetor)
R5	receita número 5 (lista completa no capítulo 7)
passo	1/60 s de lógica (<code>game_tick</code>). Todos os tempos do jogo são contados em passos
quadro	uma imagem desenhada (<code>game_draw</code>). Pode durar 1, 2 ou mais passos
ONE	4096 = 1,0 em ponto fixo (capítulo 4)
unidade	unidade de mundo: 256 = 1 metro = 1 célula do mapa

DICA Depois de **qualquer** mudança: `./dev build` (compila) e `./dev run` (compila e abre o emulador). Erros aparecem com o caminho real do arquivo e são clicáveis no terminal do VS Code.

Nos trechos de código, linhas longas continuam na linha seguinte marcadas com ↵ — no arquivo real é uma linha só. Tabs aparecem como 4 espaços.

2. Ambiente e ciclo de trabalho

O ambiente completo está no [README.md](#) (seções *Configurando o ambiente* e *Compilando*). Este capítulo resume o que um desenvolvedor usa no dia a dia.

2.1 O script `./dev`

Comando	O que faz
<code>./dev build</code>	compila dentro do Docker (imagem <code>psx-arena-sdk</code>). Gera <code>build/arena.exe</code> , <code>.bin</code> e <code>.cue</code>
<code>./dev run</code>	compila e abre no emulador configurado em <code>dev.conf</code>
<code>./dev watch</code>	recompila sozinho quando um arquivo muda
<code>./dev models</code>	exporta <code>assets/blender/*.blend</code> (coleção PSX) para <code>models/*.h</code> usando o Blender do Windows
<code>./dev vram</code>	mostra onde cada textura ficou na VRAM
<code>./dev clean</code>	apaga <code>build/</code> (use quando algo gerado parecer velho)
<code>./dev shell</code>	abre um terminal dentro do contêiner do SDK
<code>./dev doctor</code>	diagnostica Docker, emulador e caminhos
<code>VERBOSE=1 ./dev build</code>	mostra a saída completa do compilador e do linker

2.2 O que entra no build sozinho

O `CMakeLists.txt` usa `globs` com `CONFIGURE_DEPENDS`. Você **não precisa editar o CMake** para:

- um novo `src/*.c` — é compilado e ligado automaticamente;
- um novo `models/<nome>.h` — vira o símbolo `<nome>_mesh`, declarado em `build/gen/assets_gen.h`;
- um novo `assets/textures/<nome>.png` — vira um TIM em `build/textures/` e o símbolo `tex_<nome>_t` (um `TEXTURE` já carregado na VRAM por `assets_load_textures()`);
- um `.tim` pronto em `assets/tim/` — embutido como está.

O CMake só precisa ser editado para tipos novos de arquivo (por exemplo, sons — receita R8) ou para colocar arquivos soltos no CD (R9).

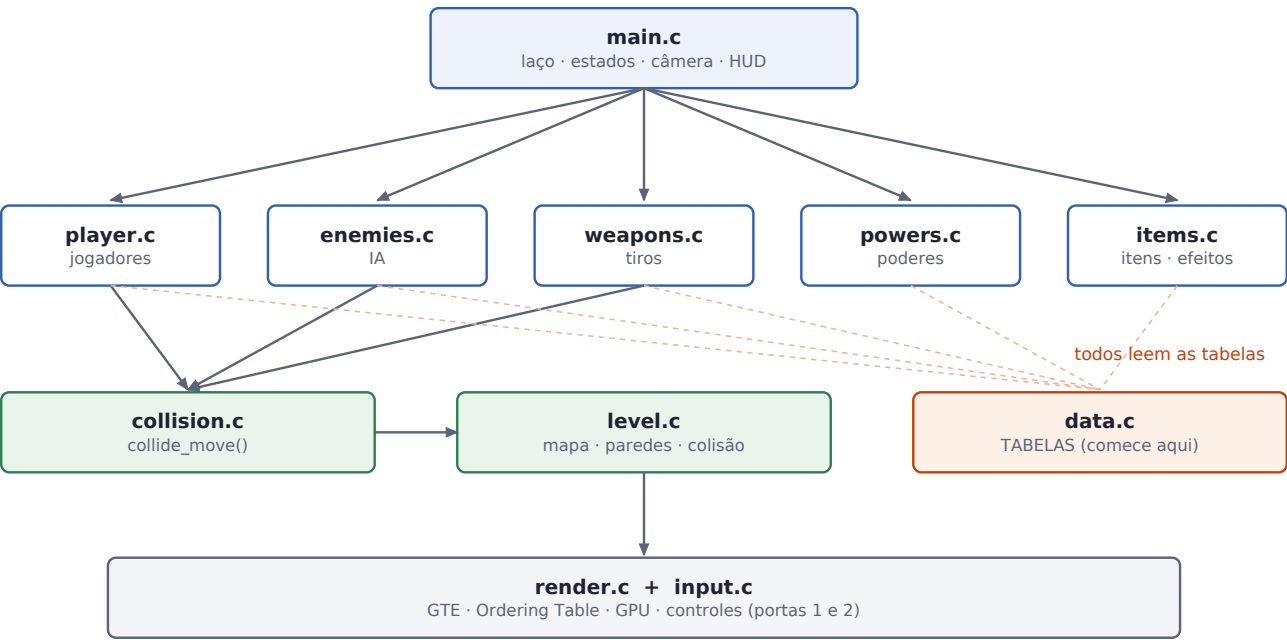
2.3 Ciclo recomendado

- Mude primeiro **tabelas** (`src/data.c`) e **constantes** (`src/config.h`).
- Se a tabela não basta, mude a **lógica** do módulo (`enemies.c`, `weapons.c`...).
- Só mexa no **motor** (`render.c`, `collision.c`, `level.c`) quando o recurso exigir.
- `./dev build` sem avisos novos → `./dev run` → teste o que mudou e o que estava perto.
- Faça *commit* pequeno, com mensagem que diga o porquê (capítulo 20).

ATENÇÃO O arquivo `dev` precisa ter fim de linha LF. O `.gitattributes` garante isso no Git; se você copiar o projeto por outro meio e aparecer `bad interpreter`, rode `sed -i 's/\r$//' dev`.

3. Arquitetura do jogo

3.1 Mapa dos módulos



Quem chama quem. As tabelas de `data.c` são lidas por todos os módulos de jogo.

Arquivo	Responsabilidade	Funções públicas principais
<code>main.c</code>	laço com passo fixo, máquina de estados, câmera, HUD, tela de seleção	<code>main</code> , <code>show_message</code> (as demais são <code>static</code>)
<code>data.c</code>	tabelas: personagens, armas, poderes, inimigos, skins, cenário	— (só dados)
<code>config.h</code>	constantes de ajuste	—
<code>game.h</code>	todos os tipos, o estado global <code>g</code> e os protótipos	—
<code>level.c</code>	mapa em texto → objetos, geometria (blocos 4x4) e colisão com células	<code>level_load</code> , <code>level_draw</code> , <code>level_blocked</code> , <code>level_cell_solid</code> , <code>level_set_solid</code>
<code>player.c</code>	até 2 jogadores, controle, dano, respawn	<code>player_spawn</code> , <code>player_update</code> , <code>player_damage</code> , <code>player_draw</code> , <code>player_nearest</code>
<code>enemies.c</code>	IA (perseguir/vagar), dano por contato, morte e drops	<code>enemy_spawn</code> , <code>enemies_update</code> , <code>enemy_damage</code> , <code>enemy_nearest_in_cone</code>
<code>weapons.c</code>	disparo (com mira automática) e projéteis	<code>weapon_fire</code> , <code>bullets_update</code> , <code>bullets_draw</code>
<code>powers.c</code>	poderes (gastam energia)	<code>power_use</code> , <code>power_shockwave</code> , <code>power_dash</code> , <code>power_heal</code>
<code>items.c</code>	itens, caixas, efeitos, sombras, objetos de cenário	<code>pickup_spawn</code> , <code>crate_damage</code> , <code>effect_spawn</code> , <code>prop_spawn</code> , <code>shadow_draw</code>
<code>collision.c</code>	colisão por círculos entre tudo	<code>collide_move</code> , <code>collide_blocked</code> , <code>collide_prop_at</code>
<code>render.c</code>	motor 3D: GTE, Ordering Table, primitivas, texto	<code>render_mesh</code> , <code>render_set_camera</code> , <code>render_rect</code> , <code>hud_print</code> , <code>render_end_frame</code>
<code>input.c</code>	lê os controles das portas 1 e 2	<code>input_init</code> , <code>input_update</code>

Arquivo	Responsabilidade	Funções públicas principais
<code>mathutil.c</code>	ângulos, distância, aleatório	<code>angle_of</code> , <code>angle_diff</code> , <code>turn_towards</code> , <code>dist2d</code> , <code>rand_range</code>

3.2 O laço principal e o passo fixo

O PS1 desenha a 60 Hz (NTSC). Quando a cena é pesada, um quadro pode levar 2 ou 3 retrazos. Se a lógica rodasse uma vez por quadro, o jogo ficaria em câmera lenta. Por isso a lógica (`game_tick`) roda em **passos fixos de 1/60 s**, quantos forem necessários para alcançar o tempo real, e o desenho (`game_draw`) roda uma vez por quadro:

`src/main.c — main()`

```
int last_vsync = VSync(-1);

while (1) {
    /* PASSO FIXO: a lógica sempre roda 60 vezes por segundo, mesmo que
     * o desenho caia para 30 ou 20 quadros. Assim o jogo não fica
     * "em câmera lenta" quando a cena é pesada. */
    int now = VSync(-1);
    int ticks = now - last_vsync;
    last_vsync = now;
    if (ticks < 1) ticks = 1;
    if (ticks > 4) ticks = 4;

    input_update(g.in);
    if (g.in[0].pressed & PAD_L2) /* L2 liga/desliga a depuração */
        show_debug ^= 1;

    for (int t = 0; t < ticks; t++) {
        game_tick();
        for (int i = 0; i < MAX_PLAYERS; i++)
            g.in[i].pressed = 0; /* "apertou agora" vale só 1 passo */
    }

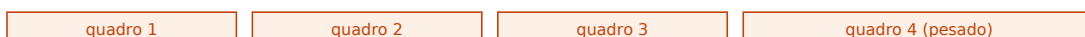
    game_draw();
    render_end_frame();
}
```

VSync (60 Hz)

`game_tick()`



`game_draw()`



Quadro que leva 2 VSyncs → o laço roda 2 passos antes de desenhar de novo; com 3 VSyncs, 3 passos.
A velocidade do jogo não muda com o FPS (limite: 4 passos por quadro).

Passos de lógica × quadros desenhados.

POR QUÊ? Passo fixo traz três vantagens: (1) velocidade do jogo independente do FPS; (2) todos os tempos podem ser contados em passos inteiros ($60 = 1$ segundo); (3) a simulação fica **determinística** — com as mesmas entradas, o mesmo resultado. O modo online por lockstep (capítulo 17) depende disso.

`g.in[i].pressed` ("acabou de apertar") é zerado depois do primeiro passo, para um toque não valer duas vezes quando o laço roda vários passos no mesmo quadro.

3.3 Máquina de estados

`g.state` decide o que `game_tick()` e `game_draw()` fazem. Cada estado tem um `case` nas duas funções:



Estados do jogo (enum em `game.h`).

src/main.c — game_tick(), estado de jogo

```

case STATE_PLAY:
    if (any_pressed(PAD_START) >= 0) {
        g.state = STATE_PAUSE;
        break;
    }
    for (int i = 0; i < MAX_PLAYERS; i++)
        player_update(&g.players[i], &g.in[i]);
    enemies_update();
    bullets_update();
    pickups_update();
    effects_update();
    camera_update(0);
    g.play_frames++;
    if (g.enemies_left == 0 && g.state == STATE_PLAY) {
        g.state = STATE_WIN;
        for (int i = 0; i < MAX_PLAYERS; i++)
            if (player_alive(&g.players[i]))
                g.score += g.players[i].hp * 10;
        g.score += g.gems * 100;
    }
  
```

Note a **ordem** das atualizações: jogadores → inimigos → tiros → itens → efeitos → câmera. Mudar a ordem muda o resultado (por exemplo, um tiro criado neste passo só se move a partir do próximo). Para criar um estado novo, veja a receita R12 no capítulo 14.

3.4 Um quadro de desenho

src/main.c — draw_world()

```

static void draw_world(int with_players) {
    level_draw();
    props_draw();
    crates_draw();
    pickups_draw();
    enemies_draw();
    bullets_draw();
    effects_draw();
    if (with_players)
        for (int i = 0; i < MAX_PLAYERS; i++)
            player_draw(&g.players[i]);
}
  
```

- `camera_apply()` envia a câmera calculada na lógica para a GTE (`render_set_camera`).
- Cada `*_draw()` chama `render_mesh()` para cada objeto; os polígonos vão para a **Ordering Table** ordenados pela profundidade (capítulo 4).

- O HUD (`hud_print`, `render_rect`) entra nas posições 0 e 1 da OT, sempre por cima.
- `render_end_frame()` espera o desenho anterior, troca os buffers, aguarda o VSync e manda a GPU desenhar a OT.

3.5 Estado global e pools

Todo o estado da partida mora em **uma** variável, `GAME g` (declarada em `game.h`). Entidades ficam em **vetores de tamanho fixo** com uma flag `active` — não existe `malloc`:

src/game.h — GAME

```
typedef struct {
    int     state;
    int     frame;
    int     play_frames;    /* tempo de jogo (para o placar) */
    int     score;
    int     gems, gems_total;
    int     enemies_left;

    INPUT   in[MAX_PLAYERS];    /* controle de cada porta */
    PLAYER  players[MAX_PLAYERS];
    int     num_players;        /* 1 ou 2 */
    int     spawn_x, spawn_z;   /* posição do 'P' no mapa */
    ENEMY   enemies[MAX_ENEMIES];
    BULLET  bullets[MAX_BULLETS];
    PICKUP  pickups[MAX_PICKUPS];
    CRATE   crates[MAX_CRATES];
    EFFECT  effects[MAX_EFFECTS];
    PROP    props[MAX_PROPS];

    int     cam_yaw;           /* ângulo da câmera ao redor dos jogadores */
    VECTOR  cam_pos;           /* posição suavizada da câmera */

    char    message[40];       /* mensagem temporária no centro da tela */
    int     message_timer;

} GAME;
```

Criar uma entidade = procurar o primeiro slot com `active == 0`. Destruir = `active = 0`. Se o pool estiver cheio, a criação é ignorada em silêncio (um tiro a menos não quebra nada). Os tamanhos ficam em `config.h` (`MAX_ENEMIES`, `MAX_BULLETS`...). Como `game_reset()` faz `memset(&g, 0, ...)`, **zero precisa ser um valor inicial válido** para todo campo novo que você criar.

3.6 Regras de ouro do projeto

Regra	Motivo
Sem <code>float/double</code>	o R3000 não tem FPU; ponto flutuante é emulado e muito lento
Sem <code>malloc</code>	2 MB de RAM, sem fragmentação; tudo em pools estáticos
<code>*_update()</code> não desenha; <code>*_draw()</code> não muda o jogo	a lógica pode rodar várias vezes por quadro; o desenho, uma
Todo movimento passa por <code>collide_move()</code>	ninguém atravessa ninguém; regra única de colisão
Funções de jogador recebem <code>PLAYER *p</code>	funciona igual para o jogador 1 e o 2; nunca use um "jogador global"
Prefira tabelas em <code>data.c</code>	mudar o jogo sem mexer no motor; menos bugs
Texto da tela em ASCII sem acentos	a fonte embutida do PSn00bSDK só tem ASCII
<code>rand()</code> só na lógica	o desenho não pode consumir números aleatórios (quebraria o determinismo e o modo link)

4. O PlayStation 1 para quem vem de back-end

Quatro conceitos explicam 90% das decisões do código. Vale ler antes de mexer em movimento, câmera ou desenho.

Componente	Especificação	Consequência no código
CPU	MIPS R3000A, 33,8 MHz, sem FPU	ponto fixo em tudo; evitar divisões em laços
GTE	coprocessador de geometria (matrizes, projeção, luz)	<code>render_mesh</code> usa <code>gte_rtpt</code> , <code>gte_nclip</code> , <code>gte_avsz3...</code>
GPU	2D: desenha triângulos/quads já projetados; sem Z-buffer	ordem de desenho pela Ordering Table
RAM	2 MB	pools estáticos; modelos e texturas embutidos no executável
VRAM	1 MB (1024×512 × 16 bits)	telas, texturas e paletas dividem o mesmo espaço
SPU	24 vezes ADPCM, 512 KB de RAM de som	sons em formato VAG (R8)
CD-ROM	2× (~300 KB/s)	o executável inteiro é carregado no boot; dados extras via <code>psxcd</code> (R9)

4.1 Ponto fixo

Sem FPU, números fracionários são inteiros escalados: **ONE = 4096 representa 1,0**. Assim, `ONE * 3 / 2` = 1,5 e `ONE / 4` = 0,25. A regra prática: **multiplicou dois valores em ponto fixo, desloque 12 bits à direita**.

ponto fixo na prática

```
int escala = ONE * 3 / 2;           /* 1,5 */
int raio   = (90 * escala) >> 12;  /* 90 x 1,5 = 135 */

/* seno/cosseno do PSn00bSDK: entrada 0..4095 (= 0..360 graus), saída -4096..4096 */
int vx = (isin(angle) * speed) >> 12; /* componente X de uma velocidade */
int vz = (icos(angle) * speed) >> 12;
```

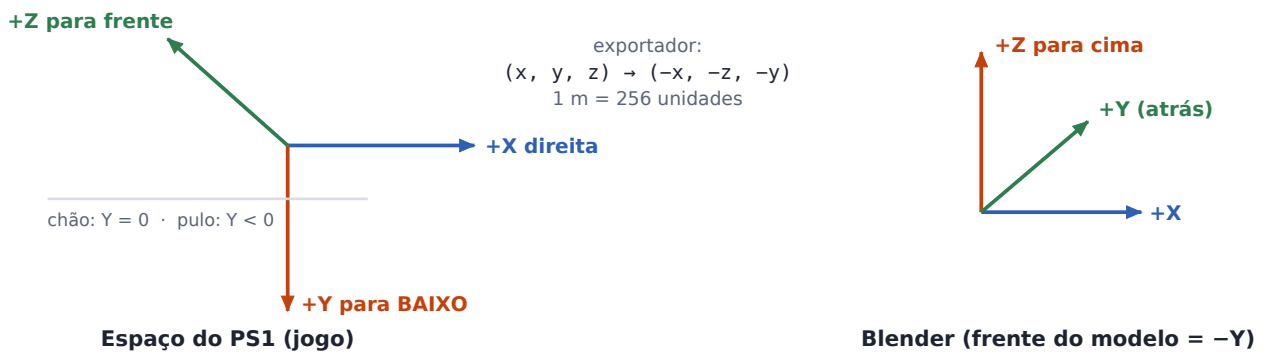
ATENÇÃO Estouro de inteiro. `int` tem 32 bits. Coordenadas do mapa chegam a ~8 000; `8000 * 8000` = 64 milhões (cabe), mas `8000 * 8000 * 4096` não cabe. Ao multiplicar três termos, desloque antes `((a * b) >> 12) * c`. Distâncias usam `dist2d()`, que evita quadrados.

4.2 Ângulos e trigonometria

Ângulos usam **4096 = 360°** (1024 = 90°, 512 = 45°). O ângulo 0 aponta para **+Z** e 1024 para **+X**, porque o código usa `x += isin(a)` e `z += icos(a)`. `mathutil.c` oferece:

Função	Uso
<code>angle_of(dx, dz)</code>	atan2 aproximado: direção de um vetor (0..4095). Ex.: mirar num inimigo
<code>angle_diff(a, b)</code>	diferença de ângulos em -2048..2047 (o caminho mais curto)
<code>turn_towards(a, alvo, taxa)</code>	gira <code>a</code> em direção a <code>alvo</code> , no máximo <code>taxa</code> por chamada (curvas suaves)
<code>dist2d(dx, dz)</code>	distância aproximada (erro de poucos %), sem raiz quadrada
<code>rand_range(lo, hi)</code>	inteiro aleatório em [lo, hi], usando <code>rand()</code>

4.3 Eixos e unidades



Eixos do PS1 e do Blender. O exportador faz a conversão.

- **Y aponta para baixo.** O chão é $Y = 0$; pular deixa `pos.vy` **negativo**. Gravidade **soma** em `vy`.
- **256 unidades = 1 metro = 1 célula do mapa** (`TILE_SIZE`). Uma célula (`cx, cz`) tem centro em `cx * 256 + 128`.
- **VECTOR** (32 bits por eixo) guarda posições no mundo; **SVECTOR** (16 bits) guarda vértices e rotações.

4.4 GTE, Ordering Table e zbias

A GPU do PS1 não tem Z-buffer: ela desenha na ordem em que recebe. O jogo usa a **Ordering Table (OT)**: um vetor de listas encadeadas, uma por faixa de profundidade. `render_mesh()` projeta cada face com a GTE, calcula a profundidade média e insere o polígono no índice correspondente. No fim do quadro, a GPU percorre a OT do fundo para a frente (algoritmo do pintor).

Ordering Table (OT_LEN = 2048, em ordem reversa: o índice maior é desenhado primeiro)

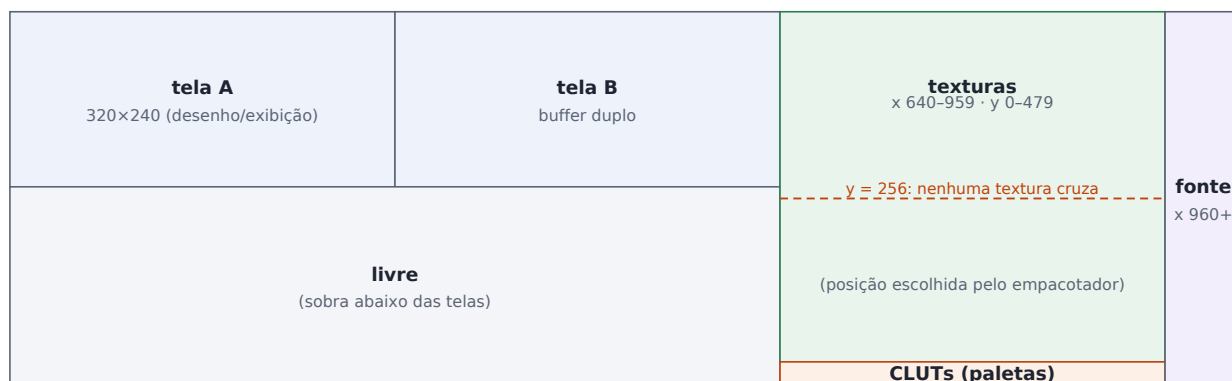


Cada polígono entra no índice = profundidade média $(avsz3/avsz4) + zbias$. Chão: +6, sombras: +2.

*A Ordering Table e o **zbias**.*

- `DRAWOPT.zbias` empurra um modelo para trás (positivo) ou para a frente (negativo) na OT. O chão usa +6 e as sombras +2, para nunca aparecerem por cima de quem está sobre eles.
- Faces **de costas** são descartadas (`gte_nclip`). Use `DRAW_NOCULL` para planos vistos dos dois lados.
- Faces que cruzam o plano próximo (`NEAR_Z`) são **descartadas, não recortadas** — por isso a câmera se afasta de paredes.
- Polígonos grandes perto da câmera distorcem a textura (o PS1 não corrige perspectiva). Subdivida o chão em células menores se isso incomodar.

4.5 VRAM e texturas



VRAM do PS1: 1024 × 512 pixels de 16 bits (1 MB). Coordenadas em pixels de 16 bits.

Divisão da VRAM neste projeto.

As duas telas (buffer duplo) ocupam x 0-639. O script `tools/build_textures.py` empacota cada PNG de `assets/textures/` na área x 640-959 sem cruzar *y = 256* (limite das páginas de textura) e grava a posição dentro do TIM. `render_load_texture()` lê essa posição e calcula *tpage*, *clut*, *u0* e *v0*. Texturas de 4 e 8 bits gastam $\frac{1}{4}$ e $\frac{1}{2}$ do espaço de uma de 16 bits e usam paletas (CLUTs) em y 480-511.

DICA `./dev vram` desenha o mapa real da VRAM do seu build. Se uma textura nova não couber, use o sufixo `-8bit` ou `-4bit` no nome do PNG ou reduza o tamanho.

4.6 Orçamento de memória

Seção	Tamanho atual (aprox.)	O que contém
<code>.text</code>	~64 KB	código
<code>.data</code> / <code>.rodata</code>	~27 KB	tabelas, mapas, modelos (<code>models/*.h</code>), texturas embutidas
<code>.bss</code>	~357 KB	estado <i>g</i> , buffers de primitivas (2 × 96 KB), OT (2 × 8 KB), geometria da fase
Total	~450 KB de 2 MB	sobra ~1,5 MB (menos a pilha e a área da BIOS)

O arquivo `build/arena.map` lista o tamanho de cada símbolo. Para ver os maiores: `./dev shell` e depois `mipsel-linux-gnu-nm --size-sort -S build/arena.elf | tail -20`.

5. Referência de tipos

Todos os tipos estão em `src/game.h` (jogo), `src/render.h` (desenho) e `src/mesh.h` (modelos). As tabelas abaixo descrevem cada campo e quem o usa.

5.1 Definições (tabelas de data.c)

CHARACTER_DEF — personagens jogáveis (`character_defs[]`)

Campo	Tipo	Descrição
<code>name, desc</code>	texto	nome e uma linha de descrição na tela de seleção (ASCII sem acentos)
<code>mesh</code>	<code>const MESH *</code>	modelo (<code>&heroi_mesh</code>)
<code>scale</code>	int	ONE = tamanho exportado
<code>use_skins</code>	0/1	1 = cores trocáveis por <code>skin_defs</code> (modelo com 5 materiais na ordem do robô)
<code>tex</code>	<code>const TEXTURE *</code>	textura do modelo (<code>&tex_nome_t</code>) ou NULL
<code>max_hp</code>	int	vida máxima
<code>speed</code>	int	velocidade de corrida (unidades por passo com o analógico no máximo)
<code>jump</code>	int	força do pulo (velocidade vertical inicial)
<code>weapon</code>	int	arma inicial (índice em <code>weapon_defs</code>)

WEAPON_DEF — armas (`weapon_defs[]`)

Campo	Descrição
<code>name</code>	nome mostrado no HUD
<code>cooldown</code>	passos entre disparos (60 = 1 s)
<code>speed</code>	velocidade do projétil (unidades por passo)
<code>damage</code>	dano por projétil
<code>pellets</code>	projéteis por disparo (espingarda: 5)
<code>spread</code>	abertura entre projéteis do leque (ângulo)
<code>jitter</code>	imprecisão aleatória (ângulo) — metralhadora
<code>life</code>	passos até o projétil sumir (alcance = <code>speed × life</code>)
<code>scale</code>	tamanho do modelo do projétil
<code>color</code>	cor do projétil e do efeito de impacto

POWER_DEF — poderes (`power_defs[]`)

Campo	Descrição
<code>name</code>	nome no HUD
<code>cost</code>	energia gasta (máximo PLAYER_MAX_ENERGY = 100)
<code>use</code>	ponteiro para <code>void f(PLAYER *p)</code> , implementada em <code>powers.c</code>

ENEMY_DEF — tipos de inimigo (`enemy_defs[]`)

Campo	Descrição
<code>name</code>	nome (usado também na barra de chefe, R2)
<code>map_char</code>	letra que cria este inimigo no mapa ('E', 'B', 'F'...)
<code>hp, speed, damage</code>	vida, velocidade ao perseguir (metade ao vagar), dano por contato
<code>scale</code>	tamanho do modelo; o raio de colisão acompanha (<code>enemy_radius</code>)
<code>score</code>	pontos ao derrotar
<code>mesh</code>	modelo
<code>palette[3]</code>	cores por material (corpo, chifres, olhos no <code>grunt</code>)

SKIN_DEF e PROP_DEF

Tipo / campo	Descrição
<code>SKIN_DEF.name, colors[5]</code>	corpo, cabeça, membros, visor, arma — para personagens com <code>use_skins</code>
<code>PROP_DEF.map_char</code>	dígito '1'... '9' usado no mapa
<code>PROP_DEF.mesh, scale, tex</code>	modelo, tamanho, textura opcional
<code>PROP_DEF.solid</code>	1 = bloqueia jogadores, inimigos e tiros (raio calculado do modelo)

5.2 Entidades (estado em g)**PLAYER (`g.players[0..1]`)**

Campo	Descrição
<code>index, active</code>	0 = jogador 1, 1 = jogador 2; <code>active</code> = está na partida
<code>pos, vy, on_ground</code>	posição (<code>pos.vy ≤ 0</code>), velocidade vertical, está no chão
<code>angle</code>	para onde olha (0..4095)
<code>hp, max_hp, energy</code>	vida, vida máxima (do personagem), energia (0..100)
<code>invuln</code>	passos de invulnerabilidade após dano (pisca)
<code>weapon, weapons_owned, cooldown</code>	arma atual, bitmask de armas liberadas, espera até o próximo tiro
<code>power</code>	poder selecionado (índice em <code>power_defs</code>)
<code>character, skin</code>	índices em <code>character_defs</code> e <code>skin_defs</code>
<code>dash_timer</code>	passos restantes do dash
<code>walk_anim</code>	acumula a distância andada (anima o balanço e, na R7, as poses)
<code>regen_timer</code>	contador da regeneração de energia
<code>respawn_timer</code>	caído no modo 2 jogadores: passos até voltar

ENEMY, BULLET, PICKUP, CRATE, PROP, EFFECT

Tipo	Campos principais
ENEMY	<code>type</code> (índice em <code>enemy_defs</code>), <code>hp</code> , <code>pos</code> , <code>angle</code> , <code>flash</code> (pisca ao levar dano), <code>kx/kz</code> (empurrão), <code>think</code> (troca de direção ao vagar), <code>hit_cooldown</code>
BULLET	<code>pos</code> , <code>vx/vz</code> , <code>life</code> , <code>damage</code> , <code>weapon</code> (índice — define cor e tamanho)
PICKUP	<code>type</code> (<code>PICK_GEM</code> , <code>PICK_HEALTH</code> , <code>PICK_ENERGY</code> , <code>PICK_WEAPON</code>), <code>pos</code>
CRATE	<code>hp</code> , <code>cx/cz</code> (célula), <code>flash</code> . A célula fica sólida enquanto a caixa existe
PROP	<code>def</code> (índice em <code>prop_defs</code>), <code>cx/cz</code> , <code>angle</code> , <code>radius</code> (calculado do modelo)

Tipo	Campos principais
EFFECT	type (FX_SHOCKWAVE, FX_BURST, FX_HEAL), pos, timer/duration, radius, color

INPUT (g.in[0..1])

src/game.h

```
typedef struct {
    uint16_t held;      /* botões pressionados agora (bits PAD_*) */
    uint16_t pressed;   /* botões que acabaram de ser apertados */
    int      lx, ly;     /* analógico esquerdo (-128..127, 0 = centro) */
    int      rx, ry;     /* analógico direito */
    int      connected;
} INPUT;
```

held e **pressed** usam os bits **PAD_*** de `<psxpad.h>` (**PAD_CROSS**, **PAD_SQUARE**, **PAD_START**..., apêndice C). Leia sempre do **INPUT** recebido — nunca do hardware — para o código funcionar com 2 jogadores e com o modo link.

5.3 Desenho e modelos

src/render.h — DRAWOPT

```
/* Opções de desenho de um modelo (pode passar NULL para o padrão) */
#define DRAW_UNLIT      0x01 /* sem iluminação: usa a cor pura */
#define DRAW_SEMITRANS  0x02 /* semitransparente (50%) */
#define DRAW_FLASH      0x04 /* pinta tudo de branco (efeito de dano) */
#define DRAW_NOCULL     0x08 /* desenha as duas faces do polígono */

typedef struct {
    const CVECTOR *palette; /* troca a cor por material: palette[face.mat] */
    int            npalette;
    const TEXTURE *tex;     /* textura para faces com UV */
    int            flags;    /* DRAW_* */
    int            zbias;    /* negativo = desenha "mais à frente" */
} DRAWOPT;
```

src/mesh.h — MESH_FACE e MESH

```
typedef struct {
    uint16_t v[4]; /* índices dos vértices (v[3] só em quads)
                   quads usam a ordem do PS1: v0 v1 / v2 v3 em "Z" */
    uint8_t  r, g, b; /* cor da face (em faces texturizadas: 128 = neutro) */
    uint8_t  flags;   /* FACE_* */
    uint8_t  mat;     /* índice do material no Blender (para trocar cores) */
    uint8_t  pad;
    uint8_t  uv[8];   /* u0,v0, u1,v1, u2,v2, u3,v3 (em texels) */
    uint16_t n;       /* índice da normal da face */
} MESH_FACE;

typedef struct {
    uint16_t nverts;
    uint16_t nfaces;
    const SVECTOR *verts;
    const SVECTOR *norms;
    const MESH_FACE *faces;
    int16_t radius; /* raio aproximado, usado para descarte */
} MESH;
```

- **palette** troca a cor de cada face pela cor do seu material (**palette[face.mat]**). É assim que skins, paletas de inimigos e itens coloridos funcionam com um único modelo.
- **FACE_UNLIT** vem de materiais com sufixo **_UNLIT** no Blender (olhos que "brilham").
- **radius** é usado no descarte por frustum: modelos fora da visão não gastam GTE.

6. Referência de funções

Funções públicas (declaradas em `game.h/render.h`). Funções `static` são internas ao arquivo.

level.c

Função	O que faz
<code>void level_load(int index)</code>	lê o mapa, cria entidades por caractere, gera geometria e colisão
<code>void level_draw(void)</code>	desenha chão e paredes (blocos de 4×4 células com descarte)
<code>int level_cell_solid(int cx, int cz)</code>	1 se a célula é parede/caixa/fora do mapa
<code>int level_blocked(int x, int z, int r)</code>	1 se um círculo de raio <code>r</code> encosta numa célula sólida
<code>void level_set_solid(int cx, int cz, int s)</code>	liga/desliga a solidez de uma célula (caixas)
<code>int level_width(void), level_height(void)</code>	tamanho do mapa em células

player.c

Função	O que faz
<code>void player_spawn(PLAYER *p, int x, int z)</code>	zera e posiciona; <code>character/skin/index/active</code> já preenchidos
<code>void player_update(PLAYER *p, INPUT *in)</code>	movimento, pulo, tiro, poderes, trocas, regeneração, respawn
<code>void player_damage(PLAYER *p, int amt, int fx, int fz)</code>	dano com empurrão a partir de (fx, fz); trata morte/game over
<code>void player_draw(PLAYER *p)</code>	modelo + sombra + anel colorido (2 jogadores)
<code>void player_draw_model(pos, rot, character, skin)</code>	só o modelo (usado também na seleção)
<code>int player_alive(const PLAYER *p), players_alive()</code>	está vivo / quantos vivos
<code>PLAYER *player_nearest(int x, int z, int *dist)</code>	jogador vivo mais próximo (alvo dos inimigos)

enemies.c

Função	O que faz
<code>void enemy_spawn(int type, int x, int z)</code>	cria um inimigo do tipo <code>type</code>
<code>void enemies_update(void)</code>	IA, movimento com colisão, dano por contato
<code>void enemy_damage(ENEMY *e, int amt, int px, int pz)</code>	dano + empurrão; na morte: pontos, efeito, drop
<code>ENEMY *enemy_nearest_in_cone(x, z, angle, cone, max)</code>	alvo da mira automática
<code>int enemy_radius(const ENEMY *e)</code>	raio de colisão (90 × escala)

weapons.c / powers.c

Função	O que faz
<code>void weapon_fire(PLAYER *p)</code>	dispara a arma atual de <code>p</code> (mira automática, leque, imprecisão)

Função	O que faz
<code>void bullets_update(void), bullets_draw(void)</code>	move projéteis; colisão com parede, caixa, cenário e inimigos
<code>void power_use(PLAYER *p)</code>	cobra energia e chama <code>power_defs[p->power].use(p)</code>

items.c

Função	O que faz
<code>void pickup_spawn(int type, int x, int z)</code>	cria um item
<code>void crate_spawn(int cx, int cz), crate_damage(cx, cz, amt)</code>	caixas destrutíveis (podem soltar itens)
<code>void effect_spawn(type, x, z, radius, dur, r, g, b)</code>	efeito visual (onda, explosão, cura)
<code>void shadow_draw(const VECTOR *pos, int scale)</code>	sombra redonda no chão
<code>void prop_spawn(int def, int cx, int cz)</code>	objeto de cenário (raio calculado do modelo)

collision.c

Função	O que faz
<code>int collide_move(VECTOR *pos, dx, dz, r, self, mask)</code>	move testando X e Z separados (desliza em paredes); 1 = bateu
<code>int collide_blocked(x, z, y, r, self, mask, fx, fz)</code>	o círculo em (x,z) colide? (só bloqueia se aproxima)
<code>int collide_prop_at(int x, int z, int r)</code>	existe objeto sólido ali? (usado pelos tiros)

render.c

Função	O que faz
<code>void render_mesh(mesh, pos, rot, scale, opt)</code>	desenha um modelo; <code>rot</code> e <code>opt</code> podem ser <code>NULL</code>
<code>void render_set_camera(eye, target)</code>	monta a matriz de visão (LookAt)
<code>void render_rect(x, y, w, h, r, g, b, semi)</code>	retângulo 2D (barras do HUD)
<code>void hud_print(x, y, fmt, ...)</code>	texto 2D estilo <code>printf</code> (ASCII, 8×8 px)
<code>void render_load_texture(tim, TEXTURE *out)</code>	envia um TIM para a VRAM (o build já faz isso para <code>assets/</code>)
<code>void render_end_frame(void)</code>	fecha o quadro, troca buffers, espera VSync
<code>int render_stats_bytes(void)</code>	bytes de primitivas usados no último quadro (máx. <code>PACKET_LEN</code>)

main.c / mathutil.c / input.c

Função	O que faz
<code>void show_message(const char *t, int frames)</code>	mensagem central temporária
<code>angle_of, angle_diff, turn_towards, dist2d, rand_range</code>	ver capítulo 4.2
<code>void input_update(INPUT in[2])</code>	lê as duas portas e calcula <code>pressed</code>

7. Receitas: visão geral e ajustes rápidos

As receitas seguem sempre a mesma estrutura: **objetivo**, **arquivos tocados**, **passos com código** e **como testar**. Elas podem ser aplicadas em qualquer ordem, exceto onde indicado.

#	Receita	Capítulo	Arquivos	Depende de
R1	Inimigo que atira	11	<code>game.h</code> , <code>weapons.c</code> , <code>enemies.c</code> , <code>data.c</code>	—
R2	Chefe com barra de vida	11	<code>game.h</code> , <code>data.c</code> , <code>main.c</code>	R1
R3	Várias fases e progressão	13	<code>game.h</code> , <code>level.c</code> , <code>render.c/.h</code> , <code>main.c</code>	—
R4	Item novo (velocidade)	12	<code>game.h</code> , <code>items.c</code> , <code>player.c</code> , <code>level.c</code>	—
R5	Arma explosiva (granada)	9	<code>game.h</code> , <code>data.c</code> , <code>weapons.c</code>	R1 (owner)
R6	Poder de congelar	10	<code>game.h</code> , <code>powers.c</code> , <code>data.c</code> , <code>enemies.c</code>	R1 (campo vizinho)
R7	Animação por poses	8	<code>game.h</code> , <code>player.c</code> , <code>main.c</code> , <code>data.c</code>	—
R8	Efeitos sonoros (SPU)	16	<code>sound.c/.h</code> (novos), <code>CMakeLists.txt</code> , <code>main.c</code> , <code>weapons.c</code>	—
R9	Ler arquivos do CD	16	<code>cdload.c</code> (novo), <code>iso.xml</code>	—
R10	Recorde no memory card	16	<code>memcard.c</code> (novo)	—
R11	Modo link (2 consoles)	17	<code>net.c/.h</code> (novos), <code>config.h</code> , <code>main.c</code>	—
R12	Novo estado: introdução da fase	14	<code>game.h</code> , <code>main.c</code>	R3

DICA A dependência "R1" de R5/R6 é só de **posição**: os trechos de `game.h` mostram os campos vizinhos criados na R1. Sem a R1, coloque o campo novo no fim da `struct` e remova a verificação de `owner`.

7.1 Ajustes por constante (config.h)

Muitas mudanças de sensação de jogo são uma linha em `src/config.h`. A lista completa está no apêndice B.

Quero...	Mude	Exemplo
câmera mais longe / mais alta	<code>CAM_DIST</code> , <code>CAM_HEIGHT</code>	<code>820 → 1100</code> ; <code>600 → 800</code>
girar a câmera mais rápido	<code>CAM_TURN_SPEED</code>	<code>40 → 64</code>
pulo mais "flutuante"	<code>GRAVITY</code>	<code>3 → 2</code>
mais zoom (FOV menor)	<code>FOV_H</code>	<code>160 → 200</code>
ver mais longe	<code>DRAW_DIST</code>	<code>4200 → 5500</code> (custa desempenho)
mais inimigos ao mesmo tempo	<code>MAX_ENEMIES</code>	<code>24 → 40</code> (memória e CPU)
respawn mais rápido (2 jogadores)	<code>RESPAWN_TIME</code>	<code>300 → 180</code> (3 s)
mais tempo invulnerável após dano	<code>PLAYER_INVULN</code>	<code>60 → 90</code>
energia volta mais rápido	<code>ENERGY_REGEN_DELAY</code>	<code>20 → 10</code>
outra cor de céu	<code>CLEAR_R/G/B</code>	R3 permite uma cor por fase

7.2 Ajustes por tabela (data.c)

Atributos de personagens, armas, poderes e inimigos são **dados**. Uma linha nova numa tabela cria um item novo sem tocar na lógica — as funções percorrem os vetores usando `num_* (sizeof(tabela) / sizeof(tabela[0]))`, que se atualiza sozinho.

src/data.c — exemplo de tabela

```
const WEAPON_DEF weapon_defs[] = {
    /* nome          recarga vel  dano proj abert. impr. vida tamanho cor */
    { "BLASTER",      10,  44,  1,  1,  0,  0,  50, ONE,  { 255, 255, 120 } },
    { "ESPINGARDA",   32,  38,  1,  5, 90, 20,  20, ONE*3/4, { 255, 150,  60 } },
    { "METRALHADORA",  4,  52,  1,  1,  0, 70,  40, ONE*2/3, { 120, 255, 255 } },
    { "CANHAO",       50,  26,  8,  1,  0,  0,  90, ONE*5/2, { 255,  80, 255 } },
};
const int num_weapons
```

ATENÇÃO Inicializadores de `struct` em C são **posicionais**. Ao acrescentar um campo novo a um tipo (como nas receitas), coloque-o **no fim** da `struct`: as linhas antigas continuam válidas e o campo novo fica zero nelas.

8. Personagens



Tela de seleção: cada linha de `character_defs` vira uma opção.

8.1 Do Blender para o jogo

O exportador `tools/blender_export_psx.py` gera um `.h` por objeto. As regras que ele espera:

Regra	Detalhe
Poucos polígonos	50 a 500 faces por personagem; o jogo inteiro desenha poucos milhares por quadro
Escala	1 m = 256 unidades. Personagens de 1,2 a 1,8 m. Propriedade <code>psx_scale</code> muda isso por objeto
Frente	vista <i>Front</i> (Numpad 1), olhando para <code>-Y</code> no Blender
Origem nos pés	a posição do objeto é ignorada; rotação e escala valem
Materiais = cores	<i>Base Color</i> do Principled BSDF. A ordem dos slots importa (paletas/skins)
<code>_UNLIT</code>	material com esse sufixo ignora a luz (olhos, luzes)
N-gons	faces com mais de 4 vértices são trianguladas pelo exportador
Limite	coordenadas de 16 bits: até ~120 m da origem
Textura	UV map + nó <i>Image Texture</i> ; o PNG é copiado para <code>assets/textures/</code> automaticamente

Fluxo recomendado: edite `assets/blender/exemplos.blend`, coloque o objeto na coleção **PSX** e rode:

terminal

```
./dev models    # cada objeto da coleção PSX vira models/<nome>.h
./dev build     # models/heroi.h -> símbolo heroi_mesh (em assets_gen.h)
```

O nome do objeto define o símbolo: **Herói Azul** → `models/heroi_azul.h` → `heroi_azul_mesh`. A propriedade personalizada `psx_name` troca o nome em C.

8.2 Adicionando o personagem

src/data.c — character_defs

```
const CHARACTER_DEF character_defs[] = {
    /* nome      descrição                modelo      tamanho skins tex  vida vel pulo
    ↪ arma */
    { "ROBO",      "EQUILIBRADO. TROCA DE SKIN",  &player_mesh, ONE,    1,    NULL, 100, 14, 34, 0
    ↪ },
    { "CAVALEIRO", "LENTO, MUITA VIDA, CANHAO",    &knight_mesh, ONE,    0,    NULL, 160, 10, 28, 3
    ↪ },
    { "BATEDOR",   "RAPIDO, POUCA VIDA, METRALHA", &scout_mesh, ONE,    0,    NULL, 70, 19, 42, 2
    ↪ },
    { "HEROI",     "MEU PERSONAGEM",                &heroi_mesh, ONE,    0,    NULL, 120, 15, 36, 1
    ↪ },
};
```

- A tela de seleção lista as linhas na ordem da tabela; não há outro registro a fazer.
- **speed** 14 e **jump** 34 são os valores "médios". Pulo 50 já passa por cima de inimigos grandes.
- **weapon** é a arma inicial; o jogador também sempre tem o BLASTER (bit 0 de **weapons_owned**).
- Com **tex** = **&tex_herói_t**, faces com UV usam a textura; as demais usam a cor do material.

8.3 Skins

Skins trocam as cores por **material**. Elas só valem para personagens com **use_skins** = 1, cujo modelo tenha 5 materiais nesta ordem: corpo, cabeça, membros, visor, arma. Para criar uma skin, acrescente uma linha em **skin_defs**:

src/data.c — skin_defs

```
{ "NEON",      { { 20, 20, 30 }, { 255, 0, 200 }, { 30, 30, 40 }, { 0, 255, 255 }, { 255, 0, 200 }
↪ } },
```

Para um personagem seu ter skins próprias, você pode criar outra tabela (ex.: **herói_skins[]**) e trocar o uso de **skin_defs** em **player_draw_model()** por uma tabela escolhida pelo personagem — por exemplo, acrescentando **const SKIN_DEF *skins; int nskins;** ao **CHARACTER_DEF**.

8.4 R7 — Animação por poses

Objetivo: alternar entre duas poses de caminhada (pernas trocadas) enquanto o personagem anda. É a técnica mais barata do PS1: vários modelos estáticos, trocados no tempo. Animação esquelética é possível, mas exige outro formato de modelo.

1. Modele as poses no Blender como objetos separados na coleção PSX (ex.: **herói_andar1**, **herói_andar2**), com os mesmos materiais do modelo parado. **./dev models** gera **herói_andar1_mesh** e **herói_andar2_mesh**.

2. Campo novo no fim de **CHARACTER_DEF**:

src/game.h

```
int      weapon;    /* arma inicial (índice em weapon_defs) */
const MESH *walk[2]; /* opcional: 2 poses de caminhada (NULL = sem) */
} CHARACTER_DEF;
```

3. Escolha da pose em **player_draw_model()** (a função ganha o parâmetro **walk_anim**):

src/player.c

```
/* Desenha o modelo de um personagem (usado no jogo e na tela de seleção).
 * walk_anim > 0 alterna entre as poses de caminhada, se existirem. */
void player_draw_model(const VECTOR *pos, const SVECTOR *rot, int character, int skin, int walk_anim)
{
    const CHARACTER_DEF *c = &character_defs[character];
    const MESH *m = c->mesh;
    if (walk_anim && c->walk[0] && c->walk[1])
        m = c->walk[(walk_anim >> 7) & 1]; /* troca de pose a cada 128 unidades andadas */
    DRAWOPT opt = { 0 };
    if (c->use_skins) {
```

src/player.c (cont.)

```

    opt.palette = skin_defs[skin].colors;
    opt.npalette = 5;
}
opt.tex = c->tex;
render_mesh(m, pos, rot, c->scale, &opt);
}

```

4. Atualize o protótipo e as chamadas: em `game.h`, o protótipo ganha `int walk_anim`; em `player_draw()` passe `p->walk_anim`; na tela de seleção (`main.c`) passe `0`.

chamadas

```

/* game.h */
void player_draw_model(const VECTOR *pos, const SVECTOR *rot, int character, int skin, int walk_anim);

/* player.c, em player_draw() */
player_draw_model(&pos, &rot, p->character, p->skin, p->walk_anim);

/* main.c, em select_draw() */
player_draw_model(&pos, &rot, s->character, s->skin, 0);

```

5. Preencha as poses do personagem (linhas sem o campo continuam sem animação):

src/data.c

```

{ "HEROI", "MEU PERSONAGEM", &heroi_mesh, ONE, 0, NULL, 120, 15, 36, 1,
  { &heroi_andar1_mesh, &heroi_andar2_mesh } },

```

POR QUÊ? `walk_anim` soma a velocidade a cada passo andando, então `(walk_anim >> 7) & 1` alterna a cada 128 unidades percorridas: a troca de pose acompanha a velocidade real, sem "patinar". Para 4 poses, use `walk[4]` e `(walk_anim >> 6) & 3`.

COMO TESTAR Escolha o personagem, ande devagar com o analógico e depois rápido: as poses devem alternar mais rápido quando corre. Parado, aparece o modelo `mesh`.

9. Armas e projéteis

9.1 Como um tiro acontece

1. `player_update()` vê QUADRADO segurado e `cooldown == 0` → chama `weapon_fire(p)`.
2. `weapon_fire()` procura um inimigo num cone à frente (`enemy_nearest_in_cone`, ~26°, 2600 unidades) e mira nele (mira automática); depois cria `pellets` projéteis em leque (`spread`) com imprecisão (`jitter`).
3. `bullets_update()` move cada projétil, testa parede/caixa/cenário e depois inimigos (caixa de colisão proporcional ao tamanho do inimigo).
4. `bullets_draw()` desenha o modelo `bullet` com a cor da arma, girando.

src/weapons.c — weapon_fire()

```
void weapon_fire(PPLAYER *p) {
    const WEAPON_DEF *w = &weapon_defs[p->weapon];

    /* mira automática: se houver inimigo à frente, mira nele */
    int aim = p->angle;
    ENEMY *target = enemy_nearest_in_cone(p->pos.vx, p->pos.vz, p->angle, AUTO_AIM_CONE,
    ↪ AUTO_AIM_DIST);
    if (target)
        aim = angle_of(target->pos.vx - p->pos.vx, target->pos.vz - p->pos.vz);

    /* sai um pouco à frente do jogador, na altura da arma */
    int sx = p->pos.vx + ((isin(p->angle) * 90) >> 12);
    int sz = p->pos.vz + ((icos(p->angle) * 90) >> 12);
    int sy = p->pos.vy + BULLET_HEIGHT;

    for (int k = 0; k < w->pellets; k++) {
        int a = aim;
        if (w->pellets > 1)
            a += ((2 * k - (w->pellets - 1)) * w->spread) / 2; /* leque centralizado */
        if (w->jitter)
            a += rand_range(-w->jitter, w->jitter);
        spawn_bullet(sx, sy, sz, a & 4095, w, p->weapon);
    }
    p->cooldown = w->cooldown;
}
```

9.2 Arma nova só com a tabela

src/data.c — weapon_defs

```
/* nome          recarga vel  dano proj abert. impr. vida tamanho cor */
{ "LASER",        6,   70,   2,   1,   0,   0,   30, ONE/2,   { 255, 40, 40 } },
{ "ESCOPETA DUPLA", 45,  40,   2,   9,  60,  30,  16, ONE*3/4, { 255, 220, 150 } },
```

- **alcance** = `vel × vida` (o LASER acima alcança 2 100 unidades ≈ 8 células).
- **DPS aproximado** = `dano × proj × 60 / recarga` (LASER: $2 \times 1 \times 60/6 = 20$ por segundo).
- Cada **W** no mapa libera a **próxima** arma da tabela que o jogador ainda não tem; a ordem importa.
- `weapons_owned` é um `uint32_t`: até 32 armas.

9.3 R5 — Arma explosiva (lança-granadas)

Objetivo: projétil que causa dano em área ao bater em parede ou inimigo.

1. Campo novo `splash` (raio da explosão) no fim de `WEAPON_DEF`:

src/game.h

```
CVECTOR  color;
int      splash;    /* raio da explosão (0 = sem explosão) */
} WEAPON_DEF;
```

2. Linha nova em `weapon_defs` (as outras armas ficam com `splash = 0`):

src/data.c

```
{ "LANCA-GRANADAS", 45, 22, 3, 1, 0, 0, 45, ONE*3/2, { 255, 200, 80 }, 520 },
```

3. Função de explosão em `weapons.c`, logo antes de `bullets_update()`:

src/weapons.c

```
/* Explosão em área: dano e empurrão em todos os inimigos no raio */
static void explode(const BULLET *b) {
    const WEAPON_DEF *w = &weapon_defs[b->weapon];
    if (b->owner || w->splash <= 0)
        return;
    effect_spawn(FX_SHOCKWAVE, b->pos.vx, b->pos.vz, w->splash, 16, 255, 160, 60);
    for (int j = 0; j < MAX_ENEMIES; j++) {
        ENEMY *e = &g.enemies[j];
        if (!e->active) continue;
        int dx = e->pos.vx - b->pos.vx, dz = e->pos.vz - b->pos.vz;
        if (dist2d(dx, dz) < w->splash) {
            int a = angle_of(dx, dz);
            enemy_damage(e, w->damage, (isin(a) * 40) >> 12, (icos(a) * 40) >> 12);
        }
    }
}

void bullets_update(void) {
```

4. Chame `explode(b)` nos dois pontos de impacto de `bullets_update()`:

src/weapons.c — bullets_update()

```
if (wall || collide_prop_at(b->pos.vx, b->pos.vz, 24)) {
    if (wall)
        crate_damage(cx, cz, b->damage);
    explode(b);                                /* <- novo */
    ...
    if (abs(dx) < r && abs(dz) < r) {
        enemy_damage(e, b->damage, b->vx / 3, b->vz / 3);
        explode(b);                            /* <- novo */
        b->active = 0;
    }
}
```

ATENÇÃO Dentro de `explode()` o inimigo atingido diretamente leva dano duas vezes (impacto + área). É intencional aqui; para evitar, passe o inimigo atingido para `explode` e pule-o no laço.

COMO TESTAR Pegue itens **W** até chegar ao LANCA-GRANADAS (é a 5ª arma). Atire num grupo: todos no raio piscam e são empurrados, com um anel laranja no chão.

9.4 Outras ideias

- **Tiro teleguiado:** em `bullets_update()`, gire `vx/vz` em direção ao inimigo mais próximo com `turn_towards()` (guarde o ângulo num campo novo do `BULLET`).
- **Munição:** campo `ammo` no `PLAYER` (um contador por arma) e um item que recarrega.
- **Tiro carregado:** conte quantos passos `QUADRADO` ficou segurando e aumente `damage/scale` ao soltar (`pressed` não serve para soltar; compare `held` com o do passo anterior).

10. Poderes

Um poder é uma função `void f(PLAYER *p)` mais uma linha em `power_defs. power_use()` verifica e cobra a energia; o botão R2 alterna o poder e o CÍRCULO usa.

src/powers.c

```
void power_use(PLAYER *p) {
    const POWER_DEF *pw = &power_defs[p->power];
    if (p->energy < pw->cost) {
        show_message("SEM ENERGIA", 40);
        return;
    }
    p->energy -= pw->cost;
    pw->use(p);
}
```

10.1 Poder simples

poder em duas partes

```
/* powers.c */
void power_super_pulo(PLAYER *p) {
    p->vy = -70; /* Y negativo = para cima */
    p->on_ground = 0;
}

/* data.c */
void power_super_pulo(PLAYER *p);
{ "SUPER PULO", 15, power_super_pulo },
```

10.2 R6 — Congelar inimigos

Objetivo: inimigos num raio ficam parados, inofensivos e azulados por 3 segundos.

1. Estado novo no inimigo (campo `frozen`, em passos):

src/game.h

```
int    fire_timer; /* contagem até o próximo tiro */
int    frozen;     /* quadros restantes congelado */
} ENEMY;
```

2. O poder:

src/powers.c

```
/* Congelar: inimigos num raio ficam parados (e inofensivos) por 3 s */
void power_freeze(PLAYER *p) {
    effect_spawn(FX_SHOCKWAVE, p->pos.vx, p->pos.vz, 1200, 26, 160, 220, 255);
    for (int i = 0; i < MAX_ENEMIES; i++) {
        ENEMY *e = &g.enemies[i];
        if (!e->active) continue;
        if (dist2d(e->pos.vx - p->pos.vx, e->pos.vz - p->pos.vz) < 1200)
            e->frozen = 180;
    }
}
```

3. Registro na tabela:

src/data.c

```
void power_freeze(PLAYER *p);

const POWER_DEF power_defs[] = {
    /* nome          custo  função */
    { "ONDA DE CHOQUE", 40, power_shockwave },
    { "DASH",          20, power_dash },
    { "CURA",           50, power_heal },
    { "CONGELAR",      45, power_freeze },
};
```

4. A IA respeita o estado: no início do laço de `enemies_update()`:

`src/enemies.c — enemies_update()`

```
const ENEMY_DEF *d = &enemy_defs[e->type];
int rad = enemy_radius(e);

if (e->frozen > 0) {          /* congelado: não anda, não ataca */
    e->frozen--;
    continue;
}
```

5. Visual: paleta de gelo em `enemies_draw()`:

`src/enemies.c — enemies_draw()`

```
static const CVECTOR ice[3] = { { 150, 200, 255 }, { 200, 230, 255 }, { 255, 255, 255 } };
DRAWOPT opt = { 0 };
opt.palette = e->frozen ? ice : d->palette;
```

E zere o campo ao criar o inimigo, em `enemy_spawn()`: `e->frozen = 0;`.

POR QUÊ? O estado mora no **inimigo**, não no poder: o poder só liga o contador e cada sistema (IA, desenho) decide o que fazer com ele. É o mesmo padrão de `flash` e `invuln`. Assim, outro poder ou uma arma podem congelar também, sem duplicar lógica.

11. Inimigos

11.1 Como a IA funciona

`enemies_update()` roda uma vez por passo para cada inimigo ativo:

1. **Alvo:** o jogador vivo mais próximo (`player_nearest`). Com 2 jogadores, a atenção se divide naturalmente.
2. **Decisão:** se o alvo está a menos de `CHASE_DIST` (3600), gira em direção a ele com `turn_towards` (96 por passo). Senão, vaga: muda de direção a cada `think` passos.
3. **Movimento:** velocidade cheia perseguindo, metade vagando, mais o empurrão (`kx/kz`, que decai 25% por passo). Tudo por `collide_move(..., COL_ALL)`. Bateu perseguindo? De vez em quando tenta contornar (± 700).
4. **Ataque:** se encosta no jogador (soma dos raios + 16) e o jogador não está no alto de um pulo, causa `damage` e espera 30 passos (`hit_cooldown`).

src/enemies.c — trecho de `enemies_update()`

```
/* --- decidir direção --- */
if (chasing) {
    e->angle = turn_towards(e->angle, angle_of(p->pos.vx - e->pos.vx, p->pos.vz - e->pos.vz),
↳ 96);
} else if (--e->think <= 0) {
    e->angle = (e->angle + rand_range(-900, 900)) & 4095;
    e->think = rand_range(40, 140);
}

/* --- mover com colisão contra paredes, cenário, jogadores e outros
 *   inimigos (ninguém atravessa ninguém) --- */
int sp = chasing ? d->speed : d->speed / 2;
int mx = ((isin(e->angle) * sp) >> 12) + e->kx;
int mz = ((icos(e->angle) * sp) >> 12) + e->kz;
e->kx = e->kx * 3 / 4;      /* empurrão vai diminuindo */
e->kz = e->kz * 3 / 4;

int blocked = collide_move(&e->pos, mx, mz, rad, e, COL_ALL);
if (blocked && !chasing)
    e->angle = (e->angle + 1024 + (rand() & 1023)) & 4095;
else if (blocked && chasing && (g.frame & 31) == (i & 31))
    e->angle = (e->angle + ((rand() & 1) ? 700 : -700)) & 4095; /* contorna */

/* --- encostou no jogador? (quem está pulando por cima escapa) ---
 * Como os dois não se atravessam, "encostar" é chegar a poucas
 * unidades da soma dos raios. */
if (e->hit_cooldown > 0) e->hit_cooldown--;
if (p && dist <= rad + PLAYER_RADIUS + 16 && p->pos.vy > -140 && e->hit_cooldown == 0) {
    player_damage(p, d->damage, e->pos.vx, e->pos.vz);
    e->hit_cooldown = 30;
}
```

11.2 Inimigo novo só com a tabela

src/data.c — `enemy_defs`

```
/* nome      letra  vida vel dano tamanho pontos modelo      paleta */
{ "TANQUE",   'T',   25,   2,   40, ONE*2,   800, &grunt_mesh, { { 90, 90, 90 }, { 40, 40, 40
↳ }, { 255, 120, 0 } } },
```

Use `T` no mapa. Para um modelo próprio, troque `&grunt_mesh` pelo seu (`&aranha_mesh`). A `palette` substitui as cores dos materiais 0, 1 e 2 do modelo; para manter as cores do Blender num tipo, faça `enemies_draw()` pular a paleta quando ela estiver zerada (por exemplo, `opt.npalette = d->palette[0].r | d->palette[0].g | d->palette[0].b ? 3 : 0;`).

ATENÇÃO Letras já usadas no mapa: # . P C G H N W E B F e dígitos (cenário). As receitas usam também S, K e V. O apêndice A tem a lista completa.

11.3 R1 — Inimigo que atira

Objetivo: um tipo de inimigo que, ao perseguir, dispara projéteis vermelhos contra o jogador. Reaproveitamos o pool de projéteis com uma marca de **dono**.

1. Tipos. Dois campos novos no fim de **ENEMY_DEF**, um contador no **ENEMY** e o dono no **BULLET**:

src/game.h — ENEMY_DEF

```
CVECTOR      palette[3];/* cores por material do modelo */
int          fire_rate;  /* quadros entre tiros (0 = não atira) */
int          bullet_speed; /* velocidade do tiro do inimigo */
} ENEMY_DEF;
```

src/game.h — ENEMY

```
int          hit_cooldown;
int          fire_timer;    /* contagem até o próximo tiro */
} ENEMY;
```

src/game.h — BULLET

```
int          weapon;
int          owner;        /* 0 = jogador, 1 = inimigo */
} BULLET;
```

src/game.h — protótipos (seção weapons.c)

```
void weapon_fire(PLAYER *p);
void enemy_fire(ENEMY *e, const PLAYER *target);
```

2. Tiro do jogador marca o dono. Em **spawn_bullet()**, logo após **b->active = 1;**, acrescente **b->owner = 0;**. Depois, a função de tiro do inimigo, em **weapons.c**:

src/weapons.c

```
/* Tiro de inimigo: usa o mesmo pool de projéteis, marcado com owner = 1 */
void enemy_fire(ENEMY *e, const PLAYER *target) {
    const ENEMY_DEF *d = &enemy_defs[e->type];
    int a = angle_of(target->pos.vx - e->pos.vx, target->pos.vz - e->pos.vz);

    for (int i = 0; i < MAX_BULLETS; i++) {
        BULLET *b = &g.bullets[i];
        if (b->active) continue;
        b->active = 1;
        b->owner = 1;
        b->pos.vx = e->pos.vx; b->pos.vy = -120; b->pos.vz = e->pos.vz;
        b->vx = (isin(a) * d->bullet_speed) >> 12;
        b->vz = (icos(a) * d->bullet_speed) >> 12;
        b->life = 90;
        b->damage = d->damage / 2;
        b->weapon = 0;
        return;
    }
}
```

3. Colisão com jogadores. Em **bullets_update()**, antes do teste contra inimigos:

src/weapons.c — bullets_update()

```
/* tiro de inimigo: testa os jogadores em vez dos inimigos */
if (b->owner == 1) {
    for (int k = 0; k < MAX_PLAYERS; k++) {
        PLAYER *p = &g.players[k];
        if (!player_alive(p) || p->pos.vy < -150) continue; /* pulando escapa */
        int dx = p->pos.vx - b->pos.vx, dz = p->pos.vz - b->pos.vz;
        if (abs(dx) < PLAYER_RADIUS + 20 && abs(dz) < PLAYER_RADIUS + 20) {
            player_damage(p, b->damage, b->pos.vx - b->vx, b->pos.vz - b->vz);
            b->active = 0;
            break;
        }
    }
}
```

src/weapons.c — bullets_update() (cont.)

```

        continue;
    }

    /* acertou um inimigo? */

```

4. Cor própria em `bullets_draw()` (substitui a linha `opt.palette = &w->color;`):

src/weapons.c — bullets_draw()

```

static const CVECTOR enemy_shot = { 255, 60, 60 };
opt.palette = b->owner ? &enemy_shot : &w->color;

```

5. A IA atira. No fim do laço de `enemies_update()` (no lugar da linha do `flash`):

src/enemies.c — enemies_update()

```

/* --- atira, se o tipo de inimigo atira --- */
if (d->fire_rate && chasing && dist < 2200) {
    if (--e->fire_timer <= 0) {
        enemy_fire(e, p);
        e->fire_timer = d->fire_rate + rand_range(0, 30);
    }
}

if (e->flash > 0) e->flash--;

```

E inicialize o contador em `enemy_spawn()`, para o primeiro tiro não sair no passo 0:

src/enemies.c — enemy_spawn()

```

e->hit_cooldown = 0;
e->fire_timer = enemy_defs[type].fire_rate; /* <- novo */

```

6. O tipo novo (letra **S**): atira a cada 75 passos (+ até 30 aleatórios), projétil com velocidade 22. Os tipos antigos ficam com `fire_rate = 0` e não atiram.

src/data.c — enemy_defs

```

{ "ATIRADOR", 'S', 4, 5, 12, ONE, 200, &grunt_mesh, { { 240, 140, 40 }, { 80, 40,
  10 }, { 255, 255, 255 } }, 75, 22 },

```



Teste da R1: o ATIRADOR (laranja) mantém distância de tiro e a vida do jogador cai sem contato.

POR QUÊ? Um pool único de projéteis com `owner` é mais simples que dois pools: uma função de movimento, uma de desenho, um limite de memória. O custo é um `if` no teste de colisão.

Pulando alto (`pos.vy < -150`) o jogador desvia — a mesma regra do contato corpo a corpo.

11.4 R2 — Chefe com barra de vida

Objetivo: um inimigo grande, resistente, que atira, com nome e barra de vida no HUD.

1. Marca de chefe (depois dos campos da R1):

src/game.h — ENEMY_DEF

```
int      bullet_speed; /* velocidade do tiro do inimigo */
int      boss;         /* 1 = chefe (barra de vida na tela) */
} ENEMY_DEF;
```

2. O chefe (letra **K**): 60 de vida, 2,5× o tamanho, atira a cada 45 passos:

src/data.c — enemy_defs

```
{ "REI SOMBRIO", 'K', 60, 3, 30, ONE*5/2, 3000, &grunt_mesh, { { 40, 40, 60 }, { 200, 170,
↳ 40 }, { 255, 40, 40 } }, 45, 18, 1 },
```

3. Barra no HUD, em `draw_hud()` (`main.c`), antes da linha de pontos:

src/main.c — draw_hud()

```
/* barra de vida do chefe, se houver um vivo */
for (int i = 0; i < MAX_ENEMIES; i++) {
    const ENEMY *e = &g.enemies[i];
    if (e->active && enemy_defs[e->type].boss) {
        const ENEMY_DEF *d = &enemy_defs[e->type];
        hud_print(160 - (int)strlen(d->name) * 4, 196, "%s", d->name);
        draw_bar(110, 208, 100, e->hp, d->hp, 255, 60, 200);
        break;
    }
}

hud_print(12, 224, "PONTOS
```



Teste da R2 (na fase 2 da R3): nome e barra do REI SOMBRIO na parte de baixo.

11.5 Mais comportamentos

- **Fugir com pouca vida:** se `e->hp < d->hp / 4`, use `angle_of` invertido (+ 2048).
- **Patrulha:** guarde dois pontos no `ENEMY` e alterne o alvo quando chegar perto.
- **Ataque em área do chefe:** a cada N passos, `effect_spawn(FX_SHOCKWAVE, ...)` e `player_damage` em quem estiver no raio (copie a lógica de `power_shockwave`, trocando inimigos por jogadores).
- **Ondas:** um contador de tempo em `game_tick` que chama `enemy_spawn(tipo, x, z)` em pontos fixos.

12. Itens e objetos de cenário

12.1 Como os itens funcionam

Itens (**PICKUP**) usam o modelo da gema com cor e tamanho por tipo (**pickup_colors**, **pickup_scale** em **items.c**). **pickups_update()** testa a distância de cada jogador vivo e chama **collect(it, p)**, que aplica o efeito. Caixas (**CRATE**) ocupam uma célula sólida até serem destruídas e podem soltar itens.

12.2 R4 — Item de velocidade

Objetivo: item verde (**V** no mapa) que deixa o jogador 50% mais rápido por 8 segundos.

1. Tipo e estado:

src/game.h

```
enum { PICK_GEM, PICK_HEALTH, PICK_ENERGY, PICK_WEAPON, PICK_SPEED, NUM_PICKUP_TYPES };
```

src/game.h — PLAYER

```
int      regen_timer;
int      speed_timer; /* quadros restantes do bônus de velocidade */
```

2. Aparência (as duas tabelas têm uma entrada por tipo):

src/items.c

```
{ 255, 255, 255 }, /* PICK_WEAPON */
{ 80, 255, 80 }, /* PICK_SPEED */
};
static const int pickup_scale[NUM_PICKUP_TYPES] = { ONE, ONE * 3 / 4, ONE * 3 / 4, ONE * 3 / 2, ONE };
```

3. Efeito ao coletar, em **collect()**:

src/items.c — collect()

```
case PICK_SPEED:
    p->speed_timer = 60 * 8; /* 8 segundos */
    show_message("VELOCIDADE!", 40);
    break;
case PICK_WEAPON: {
```

4. Efeito no movimento e contagem regressiva, em **player_update()**:

src/player.c — player_update()

```
int speed = (cdef(p)->speed * mag) / 127;
if (p->speed_timer > 0)
    speed = speed * 3 / 2; /* bônus: +50% */
```

src/player.c — fim de player_update()

```
if (p->speed_timer > 0) p->speed_timer--;
if (p->invuln > 0) p->invuln--;
```

5. Letra no mapa, em **level_load()**:

src/level.c — level_load()

```
case 'W': pickup_spawn(PICK_WEAPON, wx, wz); break;
case 'V': pickup_spawn(PICK_SPEED, wx, wz); break;
```

DICA **player_spawn()** zera o **PLAYER** campo a campo; como **game_reset()** já zera tudo com **memset**, o bônus some ao recomençar. Se quiser que o respawn também tire o bônus, acrescente **p->speed_timer = 0;** em **player_spawn()**.

12.3 Objetos de cenário (props)

Pilares, árvores, estátuas: modelo + linha em `prop_defs` + dígito no mapa. O raio de colisão é calculado dos vértices do modelo em `prop_spawn()`.

src/data.c — prop_defs

```
const PROP_DEF prop_defs[] = {  
    /* letra  modelo      tamanho  sólido  textura */  
    { '1',    &pillar_mesh,  ONE,      1,      NULL },  
    { '2',    &crate_mesh,   ONE/2,   0,      &tex_crate_t }, /* caixinha decorativa */  
    { '3',    &arvore_mesh,  ONE,      1,      &tex_folhas_t }, /* exemplo: modelo e textura seus */  
};
```

- `sólido = 1` bloqueia jogadores, inimigos e projéteis; `0` é só decoração.
- O ângulo de cada objeto é sorteado em `prop_spawn()`; para objetos que precisam de orientação fixa (cercas), troque por `0` ou por uma regra baseada na célula.
- Para algo destrutível, prefira o padrão das caixas (`CRATE`) — a célula sólida some quando `hp` chega a 0.

13. Fases

13.1 O mapa em texto

A fase é um vetor de strings em `src/level.c`. Cada caractere é uma célula de 256×256 unidades. `level_load()` lê o texto, cria entidades e gera geometria e colisão:

src/level.c — level_load()

```
for (int z = 0; z < lh; z++)
for (int x = 0; x < lw; x++) {
    char ch = grid[z][x];
    int wx = x * TILE_SIZE + TILE_SIZE / 2;
    int wz = z * TILE_SIZE + TILE_SIZE / 2;

    solid[z][x] = (ch == '#' || ch == ' ');

    switch (ch) {
        case 'P': g.spawn_x = wx; g.spawn_z = wz; break; /* jogadores nascem aqui */
        case 'C': crate_spawn(x, z); break;
        case 'G': pickup_spawn(PICK_GEM, wx, wz); break;
        case 'H': pickup_spawn(PICK_HEALTH, wx, wz); break;
        case 'N': pickup_spawn(PICK_ENERGY, wx, wz); break;
        case 'W': pickup_spawn(PICK_WEAPON, wx, wz); break;
        default:
            for (int t = 0; t < num_enemy_types; t++)
                if (enemy_defs[t].map_char == ch)
                    enemy_spawn(t, wx, wz);
            for (int t = 0; t < num_props; t++)
                if (prop_defs[t].map_char == ch)
                    prop_spawn(t, x, z);
            break;
    }
}

build_geometry();
```

- Todas as linhas com o **mesmo comprimento**; máximo **32×32** (`MAX_W`, `MAX_H`).
- Espaço em branco é vazio sólido (sem chão). `#` gera parede com dois "andares" de textura.
- Chão e paredes são agrupados em blocos de 4×4 células, descartados quando fora da visão.
- O jogador 2 nasce na primeira célula livre ao lado do `P`.

13.2 R3 — Várias fases e progressão

Objetivo: uma lista de fases, cada uma com mapa, texturas e cor de céu próprios. Vencer leva à próxima; os pontos acumulam.

1. O tipo da fase, em `game.h` (depois de `extern GAME g;`, porque usa `TEXTURE`):

src/game.h

```
/* Uma fase: mapa em texto + aparência */
typedef struct {
    const char      *name;
    const char *const *map; /* linhas do mapa, terminadas em NULL */
    const TEXTURE   *floor; /* textura do chão */
    const TEXTURE   *wall;  /* textura das paredes */
    uint8_t         sky_r, sky_g, sky_b; /* cor de fundo */
} LEVEL_DEF;

extern const LEVEL_DEF level_defs[];
extern const int      num_levels;
```

2. Os mapas viram vetores independentes terminados em `NULL`. Renomeie o vetor atual `level_maps[][32]` para `map_arena[]` (só as linhas da fase 1 + `NULL`) e acrescente as novas fases e a tabela:

src/level.c

```
static const char *const map_arena[] = {
    "#####",
    "#P.....#.....C.....#",
    /* ... resto das linhas da fase 1 ... */
    "#####",
    NULL
};

static const char *const map_ruinas[] = {
    "#####",
    "#P....#.....1...G#",
    "#..C...#..S.....#",
    "#.....#...####...#",
    "#...E.....#...E..#",
    "#####.1...#W.#....#",
    "#.....#...####...#",
    "#..G...B.....#...",
    "#.....####...C...#",
    "#..H...#...#...1...#",
    "#.....#..K.#.....#",
    "#.....#...#...#...",
    "#..N...S...V..G...#",
    "#####",
    NULL
};

const LEVEL_DEF level_defs[] = {
    /* nome      mapa      chão      parede      céu (R, G, B) */
    { "ARENA",    map_arena, &tex_floor_t, &tex_wall_t, 20, 24, 48 },
    { "RUINAS",   map_ruinas, &tex_wall_t, &tex_floor_t, 60, 30, 24 },
};

const int num_levels = sizeof(level_defs) / sizeof(level_defs[0]);
```

3. Cor de fundo por fase, em `render.h/render.c`:

src/render.h

```
void render_end_frame(void);

/* Cor de fundo (céu) — muda por fase */
void render_set_clear_color(int r, int g, int b);
```

src/render.c (antes de `render_stats_bytes`)

```
void render_set_clear_color(int r, int g, int b) {
    for (int i = 0; i < 2; i++)
        setRGB0(&fb[i].draw, r, g, b);
}

int render_stats_bytes(void) {
```

4. `level_load()` e `level_draw()` usam a tabela:

src/level.c — início de `level_load()`

```
static int cur_level = 0;

void level_load(int index) {
    const LEVEL_DEF *ld = &level_defs[index];
    const char *const *map = ld->map;
    cur_level = index;
    render_set_clear_color(ld->sky_r, ld->sky_g, ld->sky_b);
```

src/level.c — `level_draw()`

```
fo.tex = level_defs[cur_level].floor;
fo.zbias = 6; /* chão sempre atrás dos objetos sobre ele */
wo.tex = level_defs[cur_level].wall;
```

5. Progressão em `main.c`. Um índice de fase; `game_reset()` carrega a fase atual e preserva os pontos quando vem de uma vitória:

`src/main.c — game_reset()`

```
static int current_level = 0;          /* fase atual (índice em level_defs) */

static void game_reset(void) {
    INPUT in[MAX_PLAYERS];
    int score = (g.state == STATE_WIN) ? g.score : 0; /* pontos passam de fase */
    memcpy(in, g.in, sizeof(in));
    memset(&g, 0, sizeof(g));
    memcpy(g.in, in, sizeof(in));
    g.score = score;

    level_load(current_level);
}
```

`src/main.c — game_tick(), estados WIN/DEAD`

```
if (any_pressed(PAD_START) >= 0) {
    if (g.state == STATE_WIN) /* venceu: próxima fase (ou volta à 1ª) */
        current_level = (current_level + 1) % num_levels;
    game_reset();
    g.state = STATE_PLAY;
    show_message(level_defs[current_level].name, 120);
} else if (any_pressed(PAD_SELECT) >= 0) { /* trocar de personagem */
    current_level = 0;
    select_enter();
}
```

`src/main.c — select_tick()`

```
if (all && sel[0].joined) {
    srand(g.frame);
    current_level = 0;
    game_reset();
}
```



Fase RUINAS (texturas trocadas, céu avermelhado) e a tela de introdução da R12.

DICA Para testar uma fase sem jogar as anteriores, mude temporariamente `current_level = 0` para o índice desejado em `select_tick()`.

Mapas grandes (32×32) com muitas paredes visíveis custam desempenho; confira o FPS com L2.

13.3 Indo além

- **Altura/rampas:** hoje o chão é plano ($Y = 0$). Um segundo vetor de caracteres com alturas por célula, usado em `build_geometry()` e na gravidade do jogador, cria plataformas.
- **Portas e chaves:** um item chave e uma célula especial que deixa de ser sólida (`level_set_solid`) quando o jogador a toca com a chave.
- **Fases fora do executável:** com a R9, o mapa pode vir de um arquivo no CD — útil quando as fases ficarem grandes ou quando você quiser editar sem recompilar.

14. HUD, telas e câmera

14.1 Desenho 2D

Função	Uso
<code>hud_print(x, y, fmt, ...)</code>	texto estilo <code>printf</code> em pixels (tela 320×240, fonte 8×8). Só ASCII
<code>draw_center(y, texto)</code>	texto centralizado (<code>main.c</code>)
<code>draw_bar(x, y, w, valor, max, r, g, b)</code>	barra com fundo (vida, energia, chefe)
<code>render_rect(x, y, w, h, r, g, b, semi)</code>	retângulo; <code>semi = 1</code> para 50% transparente
<code>show_message(texto, passos)</code>	mensagem central temporária (ex.: "GEMA!")

Coordenadas do HUD: o texto ocupa 8 px por caractere, então uma linha cabe ~38 caracteres. O HUD de cada jogador fica em `draw_player_hud(p, x)`; o jogador 2 usa `x = 168`.

14.2 R12 — Um estado novo: introdução da fase

Objetivo: antes de cada fase, mostrar "FASE 2 / RUINAS" por 2 segundos com o mundo ao fundo. É o modelo para qualquer tela nova (créditos, loja, diálogo).

1. Valor novo no `enum` (no fim, para não mudar os números existentes):

`src/game.h`

```
enum { STATE_TITLE, STATE_SELECT, STATE_PLAY, STATE_PAUSE, STATE_WIN, STATE_DEAD, STATE_INTRO };
```

2. Função de entrada em `main.c`, depois de `game_reset()`:

`src/main.c`

```
static int intro_timer = 0;

/* Começa a fase atual pela tela de introdução ("FASE 2 / RUINAS") */
static void start_level(void) {
    game_reset();
    g.state = STATE_INTRO;
    intro_timer = 120;           /* 2 segundos */
}
```

3. Lógica em `game_tick()` e **4. desenho** em `game_draw()`:

`src/main.c — game_tick()`

```
case STATE_INTRO:           /* lógica: espera o tempo ou um botão */
    effects_update();
    camera_update(0);
    if (--intro_timer <= 0 || any_pressed(PAD_START | PAD_CROSS) >= 0)
        g.state = STATE_PLAY;
    break;

case STATE_PAUSE:
```

`src/main.c — game_draw()`

```
case STATE_INTRO:           /* desenho: o mundo + o nome da fase */
    camera_apply();
    draw_world(1);
    hud_print(128, 60, "FASE %d", current_level + 1);
    draw_center(76, level_defs[current_level].name);
    break;

case STATE_PAUSE:
```

5. Use `start_level()` onde antes havia `game_reset()`; `g.state = STATE_PLAY`; (início em `select_tick()` e `START` nas telas de fim).

POR QUÊ? Separar **lógica** (`game_tick`) de **desenho** (`game_draw`) por estado mantém a regra do passo fixo: o temporizador conta passos, não quadros, então a tela dura 2 s mesmo a 30 FPS.

14.3 Câmera

`camera_update()` (lógica) calcula o centro dos jogadores vivos e a distância entre eles, recua a câmera atrás do grupo e, se o ponto cair dentro de uma parede, aproxima e sobe. A posição é suavizada (1/4 do caminho por passo). `camera_apply()` só envia o resultado para a GTE.

Mudança	Onde
distância/altura/ponto de mira	<code>CAM_DIST</code> , <code>CAM_HEIGHT</code> , <code>CAM_LOOK_UP</code> (<code>config.h</code>)
suavização mais rápida	troque <code>>> 2</code> por <code>>> 1</code> em <code>camera_update()</code>
câmera fixa (estilo Resident Evil)	em <code>camera_update()</code> , escolha <code>want</code> por região do mapa e não recue
primeira pessoa	câmera na cabeça do jogador e alvo à frente (<code>isin/icos</code> do <code>angle</code>); não desenha o próprio modelo
tremor ao levar dano	some um deslocamento pequeno e aleatório em <code>cam_pos</code> por alguns passos

15. Colisão

Tudo que ocupa espaço no chão é um **círculo** no plano X/Z. Paredes e caixas são células sólidas do mapa; objetos de cenário, jogadores e inimigos são círculos com raio.

src/collision.c — a regra central

```
static int circle_blocks(int x, int z, int r, int ox, int oz, int orad,
                        int from_x, int from_z) {
    int min = r + orad;
    int dx = x - ox, dz = z - oz;
    if (dx >= min || dx <= -min || dz >= min || dz <= -min)
        return 0; /* longe: descarte rápido */
    int d2 = dx * dx + dz * dz;
    if (d2 >= min * min)
        return 0;
    int fx = from_x - ox, fz = from_z - oz;
    return d2 < fx * fx + fz * fz; /* aproximando? bloqueia */
}
```

- **Só bloqueia se aproxima:** se dois círculos já se sobrepõem (um empurrão, um respawn), o movimento que os afasta é permitido. Isso evita entidades "grudadas" para sempre.
- **X e Z separados:** `collide_move()` testa cada eixo; quem bate de lado desliza na parede.
- **Pulo passa por cima:** diferença de altura maior que `JUMP_CLEAR` (150) ignora a colisão entre pessoas.
- **self** é o próprio objeto, ignorado no teste. `mask` escolhe contra quem testar (`COL_PLAYERS`, `COL_ENEMIES`, `COL_ALL`).

15.1 Uma entidade nova que colide

Exemplo: um NPC parado que bloqueia a passagem. Passos:

1. Crie o pool (`NPC npcs[MAX_NPCS]` em `GAME`) com `pos` e raio.
2. Mova-o (se ele andar) **sempre** com `collide_move(&n->pos, dx, dz, raio, n, COL_ALL)`.
3. Para que os **outros** colidam com ele, acrescente um bloco em `collide_blocked()` igual ao dos inimigos, e uma máscara nova (`#define COL_NPC 4`) se quiser ligar/desligar.
4. Projéteis usam `collide_prop_at()` (cenário) e testes próprios em `bullets_update()`; acrescente lá se o NPC deve parar tiros.

ATENÇÃO `collide_blocked()` percorre todos os objetos: custo $O(n)$ por teste e $\sim O(n^2)$ por passo. Com os limites atuais (24 inimigos, 64 props) é barato. Se você multiplicar os limites, considere uma grade espacial (lista de entidades por célula do mapa).

16. Som, CD e memory card

Estas três receitas abrem portas para um jogo "completo": efeitos sonoros, dados fora do executável e progresso salvo. Elas compilam com a API do PSn00bSDK 0.24; leia as notas de teste de cada uma.

16.1 R8 — Efeitos sonoros (SPU)

O SPU toca amostras **ADPCM** (formato VAG) guardadas na sua própria RAM de 512 KB, em 24 vozes. O fluxo: converter o áudio para **.vag** → embutir no executável → copiar para a RAM do SPU uma vez → tocar quando quiser.

1. Converter com o **psxavenc** (projeto WonderfulToolchain/psxavenc). Ele **não** vem na imagem Docker do projeto: baixe um binário das *releases* no GitHub ou compile (Meson + bibliotecas do FFmpeg) e rode no WSL ou no Windows:

bash

```
psxavenc -t vag -f 22050 tiro.wav assets/sounds/tiro.vag
# -t vag      amostra SPU-ADPCM com cabeçalho .vag de 48 bytes (sempre mono)
# -f 22050    taxa de amostragem (menor = menos memória; 11025 serve para efeitos)
```

2. Embutir — acrescente ao **CMakeLists.txt**, antes de **psn00b_sdk_add_cd_image()**:

CMakeLists.txt

```
# Sons: assets/sounds/*.vag -> símbolo snd_<nome> (extern const uint32_t snd_<nome>[];)
file(GLOB _vags CONFIGURE_DEPENDS ${PROJECT_SOURCE_DIR}/assets/sounds/*.vag)
foreach(_vag ${_vags})
    get_filename_component(_name ${_vag} NAME_WE)
    string(MAKE_C_IDENTIFIER "${_name}" _name)
    psn00b_sdk_target_incbin(arena PRIVATE snd_${_name} ${_vag})
endforeach()
```

3. O módulo de som (dois arquivos novos em **src/**):

src/sound.h

```
/* sound.h - Efeitos sonoros (.VAG) no SPU */
#ifndef SOUND_H
#define SOUND_H
#include <stdint.h>

typedef struct {
    uint32_t addr; /* endereço na memória do SPU */
    int rate; /* taxa de amostragem (Hz) */
} SOUND;

void sound_init(void);
void sound_load(SOUND *s, const uint32_t *vag_file);
void sound_play(const SOUND *s, int volume); /* volume 0..0x3fff */

#endif
```

src/sound.c

```
/* sound.c - Carrega .VAG para a RAM do SPU (512 KB) e toca em uma das 24 vozes */
#include <psxspu.h>
#include "sound.h"

typedef struct { /* cabeçalho do arquivo .VAG (big-endian) */
    uint32_t magic, version, interleave, size, sample_rate;
    uint16_t reserved[5], channels;
    char name[16];
} VAG_HEADER;

/* Os primeiros 4 KB do SPU são reservados e o psxspu grava uma amostra
 * vazia em 0x1000; nossas amostras começam depois disso. */
static uint32_t next_addr = 0x1010;
static int next_voice = 0;
```

src/sound.c (cont.)

```

/* Os campos do cabeçalho VAG são big-endian; o PS1 é little-endian.
 * Não use __builtin_bswap32: com este GCC ele chama uma rotina da libgcc
 * compilada para outro ABI e o jogo trava (veja o capítulo 19). */
static uint32_t be32(uint32_t v) {
    const uint8_t *b = (const uint8_t *)&v;
    return ((uint32_t)b[0] << 24) | ((uint32_t)b[1] << 16) | ((uint32_t)b[2] << 8) | b[3];
}

void sound_init(void) {
    SpuInit();
}

void sound_load(SOUND *s, const uint32_t *vag_file) {
    const VAG_HEADER *h = (const VAG_HEADER *)vag_file;
    int size = (be32(h->size) + 63) & ~63;          /* DMA em blocos de 64 */

    SpuSetTransferMode(SPU_TRANSFER_BY_DMA);
    SpuSetTransferStartAddr(next_addr);
    SpuWrite((const uint32_t *)h + 1, size);
    SpuIsTransferCompleted(SPU_TRANSFER_WAIT);

    s->addr = next_addr;
    s->rate = be32(h->sample_rate);
    next_addr += size;
}

void sound_play(const SOUND *s, int volume) {
    int ch = next_voice;
    next_voice = (next_voice + 1) % 24;          /* reveza as vozes */

    SpuSetKey(0, 1 << ch);                      /* para a voz */
    SPU_CH_FREQ(ch) = getSPUSampleRate(s->rate);
    SPU_CH_ADDR(ch) = getSPUAddr(s->addr);
    SPU_CH_VOL_L(ch) = volume;
    SPU_CH_VOL_R(ch) = volume;
    SPU_CH_ADSR1(ch) = 0x00ff;                   /* envelope desligado */
    SPU_CH_ADSR2(ch) = 0x0000;
    SpuSetKey(1, 1 << ch);                      /* toca */
}

```

4. Usar: carregar no `main()` e tocar onde o evento acontece:

src/main.c

```

#include "sound.h"

extern const uint32_t snd_tiro[]; /* gerado de assets/sounds/tiro.vag */
SOUND sfx_tiro;                  /* global: outros arquivos usam com extern */

int main(void) {
    render_init();
    input_init();
    sound_init();
    sound_load(&sfx_tiro, snd_tiro);
}

```

src/weapons.c

```

#include "sound.h"
extern SOUND sfx_tiro;          /* definido em main.c */

/* ... no fim de weapon_fire(): */
p->cooldown = w->cooldown;
sound_play(&sfx_tiro, 0x2800);

```

ATENÇÃO Não use `__builtin_bswap32` neste projeto. Com o GCC `mipsel-linux-gnu` ele vira uma chamada à `libgcc` compilada com outro ABI (*abica11s*) e o jogo **trava** na hora — foi exatamente o que aconteceu no primeiro teste desta receita. A função `be32()` faz a mesma conversão com bytes.

Teste realizado: com um `.vag` de teste o jogo roda e atira normalmente com `sound_play()`; o áudio em si não foi ouvido (emulador sem som no ambiente de teste).

- Memória do SPU: 22 050 Hz mono ADPCM $\approx$ 8 KB por segundo de som. 512 KB dão $\sim$ 60 s de efeitos.

- **Música:** a forma mais simples é **CD-DA** (faixa de áudio no CD). O `iso.xml` já traz a linha comentada `<track type="audio" ...>`; para tocar, use `CdControl(CdlPlay, ...)` da `psxcd`. Música em streaming XA ou sequenciada exige mais trabalho (veja os exemplos `sound/` do `PSn00bSDK`).

16.2 R9 — Ler arquivos do CD

Hoje tudo vai dentro do `ARENA.EXE`, carregado inteiro no boot. Quando os dados crescerem (mapas, texturas por fase, sons), leia sob demanda do CD:

1. Coloque o arquivo no CD, em `iso.xml` (nomes 8.3, maiúsculos):

iso.xml — dentro de `<directory_tree>`

```
<file name="SYSTEM.CNF" type="data" source="${PROJECT_SOURCE_DIR}/system.cnf" />
<file name="ARENA.EXE" type="data" source="arena.exe" />
<file name="FASE2.DAT" type="data" source="${PROJECT_SOURCE_DIR}/assets/cd/fase2.dat" />
```

2. Função de leitura (novo `src/cdload.c`; declare o protótipo em `game.h`):

src/cdload.c

```
/* cdload.c - Lê um arquivo inteiro do CD para um buffer */
#include <psxcd.h>
#include "game.h"

/* Chame CdInit() uma vez no início do main() antes de usar.
 * nome no formato ISO 9660: "\\FASE2.DAT;1" (barra invertida, 8.3, ";1")
 * Devolve o tamanho em bytes, ou < 0 em caso de erro. */
int cd_load(const char *name, uint32_t *buf, int max_bytes) {
    CdLFILE file;
    if (!CdSearchFile(&file, name))
        return -1; /* não encontrado */

    int sectors = (file.size + 2047) / 2048; /* setores de 2 KB */
    if (sectors * 2048 > max_bytes)
        return -2; /* buffer pequeno */

    CdControl(CdlSetloc, &file.pos, 0); /* posiciona a leitura */
    CdRead(sectors, buf, CdlModeSpeed); /* lê em velocidade dupla */
    if (CdReadSync(0, 0) < 0)
        return -3; /* erro de leitura */
    return file.size;
}
```

uso

```
CdInit(); /* uma vez, no main() */

static uint32_t buf[16 * 1024 / 4]; /* 16 KB, alinhado em 4 bytes */
int n = cd_load("\\FASE2.DAT;1", buf, sizeof(buf));
if (n < 0)
    show_message("ERRO NO CD", 120);
```

COMO TESTAR O build gera a imagem com o arquivo (conferido no `.bin`). A leitura não pôde ser validada no emulador de teste deste manual, que usa uma BIOS alternativa com o executável embutido. Teste no DuckStation (com BIOS) ou PCSX-Redux abrindo o `.cue`. Durante a leitura o jogo para; leia entre fases (na tela de introdução da R12, por exemplo).

16.3 R10 — Recorde no memory card

A BIOS oferece um sistema de arquivos simples para o cartão (**bu00**: = porta 1, **bu10**: = porta 2). Cada arquivo ocupa blocos de 8 KB; o cabeçalho de save (ícone e título que aparecem no menu do console) fica nos primeiros 512 bytes.

src/memcard.c

```
/* memcard.c - Recorde no memory card via funções de arquivo da BIOS (esboço) */
#include <psxapi.h>
#include <sys/fcntl.h>
#include <string.h>
#include "game.h"

/* Nome segue o padrão de saves: região + código + nome (até 20 caracteres) */
#define SAVE_NAME "bu00:BASLUS-00000ARENA"

static uint8_t block[8192]; /* um bloco do cartão = 8 KB */

void memcard_init(void) { /* chamar uma vez, depois de input_init() */
    InitCARD(1);
    StartCARD();
    _bu_init();
}

int memcard_save_record(int score) {
    int fd = open(SAVE_NAME, FWRITE);
    if (fd < 0) { /* não existe: cria com 1 bloco */
        fd = open(SAVE_NAME, FCREATE | (1 <= 16));
        if (fd < 0) return -1; /* sem cartão ou sem espaço */
        close(fd);
        fd = open(SAVE_NAME, FWRITE);
        if (fd < 0) return -1;
    }
    memset(block, 0, sizeof(block));
    memcpy(block + 512, &score, sizeof(score)); /* 0..511: reservado ao cabeçalho */
    int n = write(fd, block, sizeof(block)); /* múltiplos de 128 bytes */
    close(fd);
    return (n == (int)sizeof(block)) ? 0 : -1;
}

int memcard_load_record(void) {
    int score = 0;
    int fd = open(SAVE_NAME, FREAD);
    if (fd < 0) return 0;
    if (read(fd, block, 1024) == 1024)
        memcpy(&score, block + 512, sizeof(score));
    close(fd);
    return score;
}
```

ATENÇÃO Esta receita é um **esboço**: compila, mas não foi validada em execução. Para aparecer no gerenciador de memória do console com nome e ícone, preencha o cabeçalho de save (formato "SC": título em Shift-JIS, paleta e ícone 16×16) nos bytes 0-511.

Inicialize o cartão **depois** de `input_init()` e evite acessá-lo a cada quadro: grave ao terminar a fase.

17. Modo online

Este é o capítulo mais longo porque o tema exige honestidade técnica: o PlayStation 1 **não tem hardware de rede**. Não existe placa Ethernet, modem nem pilha TCP/IP no console. Tudo que é "online" para um jogo de PS1 acontece **fora** do jogo (streaming) ou **pela porta serial** (cabo link), que pode ser levada pela internet quando os dois lados são emuladores.

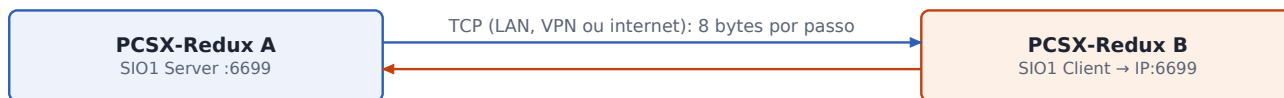
17.1 As opções

Abordagem	Como funciona	Mudança no jogo	Situação
Streaming (Parsec)	um PC roda o emulador; o convidado recebe o vídeo e envia o controle, que vira a porta 2	nenhuma (o jogo já tem 2 jogadores locais)	funciona hoje
Netplay do RetroArch	dois RetroArch sincronizam o emulador inteiro	nenhuma	o FAQ de netplay do libretro informa que PS1 não funciona
Modo link (SIO1)	cada lado roda o jogo; a cada passo os dois trocam a entrada pela porta serial (lockstep)	sim — receita R11	compila; proposta para testar (17.8)
Servidor de jogo	o console conversa com um servidor	—	impossível sem hardware de rede

A) Parsec (funciona hoje)



B) Modo link SIO1 (proposta deste manual)



Os dois caminhos viáveis.

17.2 Streaming com Parsec (sem mudar o código)

O jogo já suporta 2 jogadores na mesma tela (porta 1 e porta 2). Com streaming, o segundo controle simplesmente está em outra casa.

Quem hospeda (host)

1. Instale o emulador (PCSX-Redux ou DuckStation) e confirme que o jogo roda com `./dev run` ou abrindo o `.cue`.
2. Instale o **Parsec**, crie a conta e, nas configurações de *host*, deixe a hospedagem ativada. O Parsec instala um driver de controle virtual: cada convidado aparece para o Windows como um **controle Xinput** novo.
3. No emulador, configure a **porta 2** para usar esse controle virtual (no PCSX-Redux: *Configuration* → *Controls*; no DuckStation: *Settings* → *Controllers*, Controller Port 2). Faça isso uma vez com um convidado conectado, para o controle existir na lista.
4. Gere o convite (link de compartilhamento ou amigo no Parsec) e envie ao convidado.

Quem joga de longe (convidado)

1. Instale o Parsec, abra o convite e conecte.
2. Ligue um controle no próprio PC. Ele é enviado ao host e aparece como controle 2.
3. Na tela de título, o convidado aperta **START** no controle dele — o jogo entende como "controle 2 entrou" (a tela de seleção mostra os dois lados).

DICA Latência do streaming = rede + codificação de vídeo (tipicamente 20–60 ms em boas conexões). Para um jogo de ação de PS1 a 30 FPS isso é aceitável.

Alternativas com a mesma ideia: outras ferramentas de jogo remoto que repassam controles (por exemplo, o *Remote Play Together* da Steam com o emulador adicionado como jogo de fora da Steam). O princípio é o mesmo: o jogo só vê um segundo controle.

17.3 Modo link: a ideia

Jogos de PS1 como *Doom* e *Ridge Racer Revolution* usavam o **cabo link** (SCPH-1040), que liga a porta serial de dois consoles, cada um com sua TV e seu CD. O PSn00bSDK dá acesso a essa porta (**SIO1**) pela biblioteca **psxsio**. O PCSX-Redux consegue **emular a porta serial por TCP** (*SIO1 Server* e *SIO1 Client*): dois emuladores, na mesma máquina ou em continentes diferentes, ficam ligados como se houvesse um cabo.

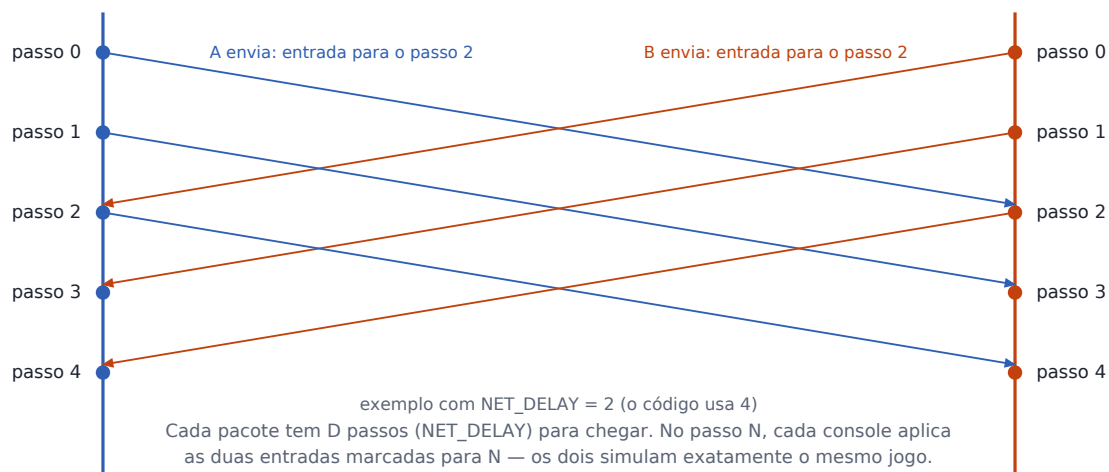
O que tráfegar pelo cabo? Duas estratégias clássicas:

Estratégia	O que envia	Para este projeto
Sincronizar o estado	posições, vida, inimigos... a cada quadro (alguns KB)	pesado para 115 200 bit/s (~11 KB/s) e exige decidir quem é a "verdade"
Lockstep (entradas)	só os botões de cada passo (8 bytes)	ideal : o jogo já é determinístico e em passo fixo; 8 bytes × 60 = 480 B/s

No **lockstep**, os dois consoles rodam a **mesma simulação**. A cada passo, cada lado envia a entrada do seu controle e espera a do outro. Com as mesmas entradas, na mesma ordem, a partir do mesmo estado inicial, os dois chegam ao mesmo resultado — sem nunca trocar posições.

CONSOLE A (host, jogador 1)

CONSOLE B (convidado, jogador 2)



Lockstep com atraso de entrada. A entrada lida no passo N vale para o passo N + D.

Atraso de entrada (NET_DELAY)

Se cada passo esperasse a resposta do outro lado, o jogo travaria por uma viagem completa da rede a cada 1/60 s. A solução é **adiar a própria entrada**: o que você aperta no passo N só vale no passo $N + D$ ($D = \text{NET_DELAY}$). Enquanto isso, o pacote viaja. Se ele chegar antes de D passos ($D \times 16,7 \text{ ms}$), o jogo nunca espera. O custo: o controle responde D passos mais tarde — **nos dois lados igualmente**, o que é justo.

NET_DELAY	Atraso sentido	Cobre latência de ida e de volta	Uso
2	33 ms	~33 ms	dois emuladores no mesmo PC ou cabo link
4 (padrão)	67 ms	~67 ms	LAN, VPN na mesma cidade
6-8	100-133 ms	~100-133 ms	internet entre cidades/países

17.4 Requisitos de determinismo

Lockstep só funciona se **as duas simulações forem idênticas, bit a bit**. Qualquer diferença cresce (um inimigo vira para outro lado, depois outro tiro acerta...) e os jogos divergem — a **dessincronia**. A tabela mostra o que o projeto já garante e o que você precisa preservar ao modificar o código:

Requisito	Situação no projeto	Cuidado ao modificar
Mesmo executável nos dois lados	—	compile uma vez e use o mesmo <code>.cue/.exe</code> nos dois
Mesma semente de <code>rand()</code>	R11: o host sorteia e envia no handshake	nunca chame <code>srand()</code> com valores locais (tempo, VSync) depois do handshake
Mesmo estado inicial	R11 chama <code>game_reset()</code> nos dois lados após conectar (zera <code>g</code> , inclusive <code>g.frame</code>)	estado fora de <code>g</code> (variáveis <code>static</code> em módulos) também precisa ser zerado ali
Lógica só depende de <code>g.in</code>	sim: <code>player_update</code> recebe <code>INPUT *</code>	nunca leia o controle direto do hardware dentro da lógica
Passo fixo, sem tempo real	sim: timers contam passos	não use <code>VSync(-1)</code> nem FPS dentro de <code>game_tick</code>
<code>rand()</code> só na lógica	sim	o desenho não pode chamar <code>rand()</code> (um lado desenha mais quadros que o outro)
Sem <code>float</code>	sim	inteiros dão o mesmo resultado em qualquer máquina
Memória inicializada	pools zerados por <code>memset</code>	inicialize todo campo novo em <code>*_spawn</code>
<code>pressed</code> sincronizado	R11 recalcula <code>pressed</code> a partir do <code>held</code> trocado	não use o <code>pressed</code> local na lógica em modo link

ATENÇÃO Coisas **locais** são permitidas: o overlay de depuração (L2), a câmera de cada um (se você a separar por jogador), sons. Elas só não podem alterar `g` de forma diferente nos dois lados. Na versão atual a câmera mora em `g` e é calculada na lógica — portanto é igual nos dois lados.

17.5 O protocolo

Handshake

1. O **host** (L1 na tela de título) envia `0xF0` + 4 bytes de semente (7 bits cada) repetidamente, até ouvir `0xF1`.
2. O **convidado** (R1) espera `0xF0`, lê a semente, responde `0xF1` e espera `0xF2`, descartando anúncios repetidos que ainda estejam chegando.
3. O host envia `0xF2` ("pacotes começam agora"). Os dois chamam `srand(semente)`, `game_reset()` e vão para a seleção com 2 jogadores.

Bytes de controle são $\geq 0x80$ e a semente viaja em bytes $< 0x80$, então nenhum byte da semente é confundido com um comando — por isso a semente usa 28 bits e não 32.

Pacote por passo (8 bytes)

Byte	Campo	Conteúdo
0	<code>sync</code>	<code>0xA5</code> — início do pacote
1	<code>frame</code>	passo a que esta entrada se refere (módulo 256)
2-3	<code>held</code>	bits dos botões (<code>PAD_*</code>)
4-5	<code>lx, ly</code>	analógico esquerdo (−128..127)
6	<code>hash</code>	resumo do estado do remetente (detecta dessincronia)
7	<code>check</code>	soma dos bytes 0-6 (detecta corrupção)

A 115 200 bit/s, 8 bytes levam ~0,7 ms. O tráfego total é ~480 bytes/s em cada sentido.

17.6 R11 — Implementação

1. Interruptor de compilação no fim de `config.h` (desligado por padrão, para o jogo normal não mudar):

src/config.h

```
/* Modo link (2 consoles pela porta serial). Descomente para compilar com ele.
 * Na tela de título: L1 = criar partida (host), R1 = entrar (convidado). */
/* #define LINK_MODE */
```

2. Interface do módulo:

src/net.h

```
/* net.h - Modo link: dois consoles/emuladores pela porta serial (lockstep) */
#ifndef NET_H
#define NET_H
#include <stdint.h>
#include "game.h"

#define NET_DELAY 4 /* atraso de entrada em passos (esconde a latência) */

void net_init(void); /* abre a serial */
/* Sincroniza os dois lados. O host escolhe a semente e envia; o convidado
 * recebe. Devolve 0 = ok, -1 = ninguém respondeu. */
int net_handshake(int is_host, uint32_t *seed);
/* Troca a entrada de UM passo. local = controle deste console.
 * Preenche out[0] (jogador 1 = host) e out[1] (jogador 2 = convidado),
 * iguais nos dois consoles. Devolve 0 = ok, -1 = conexão perdida,
 * -2 = dessincronia detectada (estados diferentes). */
int net_exchange(const INPUT *local, INPUT out[2], uint8_t state_hash);
uint8_t net_state_hash(void); /* resumo do estado do jogo */

#endif
```

3. Implementação (novo `src/net.c`; entra no build sozinho):

`src/net.c`

```

/* net.c - Lockstep pela porta serial (SI01).
 * PROPOSTA: compila, mas não foi testado em dois consoles. Teste com dois
 * PCSX-Redux ligados por "SI01 Server" / "SI01 Client" (veja o capítulo). */
#include <psxsio.h>
#include <psxapi.h>
#include <psxgpu.h>
#include "net.h"

#define NET_BAUD      115200
/* Bytes de controle são >= 0x80; a semente viaja em bytes de 7 bits
 * (< 0x80), então nunca é confundida com um byte de controle. */
#define HS_HELLO      0xF0    /* host: "estou aqui" + 4 bytes de semente */
#define HS_ACK        0xF1    /* convidado: "recebi a semente" */
#define HS_GO         0xF2    /* host: "pacotes começam agora" */
#define PKT_SYNC      0xA5    /* primeiro byte de todo pacote */
#define TIMEOUT_VBL   180    /* 3 s sem resposta = conexão perdida */

typedef struct {          /* 8 bytes por passo, nos dois sentidos */
    uint8_t sync;
    uint8_t frame;        /* passo a que esta entrada se refere (mod 256) */
    uint16_t held;        /* botões */
    int8_t lx, ly;        /* analógico esquerdo */
    uint8_t hash;         /* resumo do estado NET_DELAY passos atrás */
    uint8_t check;        /* soma de verificação */
} NET_PACKET;

static int      is_host;
static uint8_t  frame;    /* passo atual (mod 256) */
static int      steps;    /* passos desde o handshake */
static INPUT    local_q[256], remote_q[256];
static uint8_t  hash_q[256]; /* nosso hash de cada passo */
static uint16_t prev_held[2];

static uint8_t checksum(const NET_PACKET *p) {
    const uint8_t *b = (const uint8_t *)p;
    uint8_t s = 0;
    for (int i = 0; i < (int)sizeof(*p) - 1; i++) s += b[i];
    return s;
}

static void send_packet(NET_PACKET *p) {
    p->sync = PKT_SYNC;
    p->check = checksum(p);
    const uint8_t *b = (const uint8_t *)p;
    for (int i = 0; i < (int)sizeof(*p); i++)
        SIO_WriteByte(b[i]);
}

/* Lê um byte esperando no máximo 'vbl' retraços (-1 = tempo esgotado) */
static int read_byte_timeout(int vbl) {
    int start = VSync(-1);
    for (;;) {
        int c = SIO_ReadByte2();
        if (c >= 0) return c;
        if (VSync(-1) - start > vbl) return -1;
    }
}

/* Espera um pacote completo; -1 = tempo esgotado, -2 = pacote corrompido */
static int rcv_packet(NET_PACKET *p) {
    uint8_t *b = (uint8_t *)p;
    int c;
    do {                                /* procura o início do pacote */
        c = read_byte_timeout(TIMEOUT_VBL);
        if (c < 0) return -1;
    } while (c != PKT_SYNC);
    b[0] = PKT_SYNC;
    for (int i = 1; i < (int)sizeof(*p); i++) {
        c = read_byte_timeout(TIMEOUT_VBL);
        if (c < 0) return -1;
        b[i] = (uint8_t)c;
    }
}

```

src/net.c (cont.)

```

    return (checksum(p) == p->check) ? 0 : -2;
}

void net_init(void) {
    SIO_Init(NET_BAUD, MR_CHLEN_8 | MR_SB_01); /* 8 bits, 1 stop, sem paridade */
}

int net_handshake(int host, uint32_t *seed) {
    int start = VSync(-1);
    is_host = host;

    if (host) {
        *seed &= 0xffffffff; /* 4 x 7 bits */
        for (;;) { /* anuncia até alguém responder */
            SIO_WriteByte(HS_HELLO);
            for (int i = 0; i < 4; i++)
                SIO_WriteByte((*seed >> (i * 7)) & 0x7f);
            if (read_byte_timeout(30) == HS_ACK) break;
            if (VSync(-1) - start > TIMEOUT_VBL * 10) return -1;
        }
        SIO_WriteByte(HS_G0);
    } else {
        for (;;) { /* espera um anúncio válido */
            int c = read_byte_timeout(TIMEOUT_VBL * 10);
            if (c < 0) return -1;
            if (c != HS_HELLO) continue;
            uint32_t s = 0;
            int ok = 1;
            for (int i = 0; i < 4 && ok; i++) {
                int v = read_byte_timeout(30);
                if (v < 0 || v >= 0x80) ok = 0;
                else s |= (uint32_t)v << (i * 7);
            }
            if (!ok) continue;
            *seed = s;
            break;
        }
        for (;;) { /* confirma e espera o "G0" */
            SIO_WriteByte(HS_ACK);
            int c, got = 0;
            while ((c = read_byte_timeout(60)) >= 0)
                if (c == HS_G0) { got = 1; break; } /* descarta anúncios repetidos */
            if (got) break;
            if (VSync(-1) - start > TIMEOUT_VBL * 10) return -1;
        }
    }

    /* os primeiros NET_DELAY passos usam entrada vazia nos dois lados */
    for (int i = 0; i < 256; i++) {
        INPUT zero = { 0 };
        local_q[i] = remote_q[i] = zero;
        hash_q[i] = 0;
    }
    frame = 0;
    steps = 0;
    prev_held[0] = prev_held[1] = 0;
    return 0;
}

int net_exchange(const INPUT *local, INPUT out[2], uint8_t state_hash) {
    /* 1. nossa entrada vale para o passo frame + NET_DELAY: envia já */
    uint8_t target = frame + NET_DELAY;
    local_q[target] = *local;
    hash_q[frame] = state_hash;

    NET_PACKET tx = { 0 };
    tx.frame = target;
    tx.held = local->held;
    tx.lx = (int8_t)local->lx;
    tx.ly = (int8_t)local->ly;
    tx.hash = state_hash; /* hash deste passo (= target - NET_DELAY) */
    send_packet(&tx);
}

```

src/net.c (cont.) (cont.)

```

/* 2. a entrada do outro lado para ESTE passo foi enviada NET_DELAY
 * passos atrás; normalmente já está esperando no buffer */
int desync = 0;
if (steps >= NET_DELAY) {
    NET_PACKET rx;
    if (recv_packet(&rx) < 0 || rx.frame != frame)
        return -1; /* sumiu ou chegou fora de ordem */
    remote_q[frame].held = rx.held;
    remote_q[frame].lx = rx.lx;
    remote_q[frame].ly = rx.ly;
    /* o hash recebido é do passo frame - NET_DELAY do outro lado */
    desync = (rx.hash != hash_q[(uint8_t)(frame - NET_DELAY)]);
}

/* 3. entrega a entrada do passo atual, igual nos dois consoles */
const INPUT *mine = &local_q[frame], *theirs = &remote_q[frame];
out[0] = is_host ? *mine : *theirs; /* jogador 1 = host */
out[1] = is_host ? *theirs : *mine;
for (int k = 0; k < 2; k++) { /* "pressed" derivado do held sincronizado */
    out[k].rx = out[k].ry = 0;
    out[k].connected = 1;
    out[k].pressed = out[k].held & ~prev_held[k];
    prev_held[k] = out[k].held;
}
frame++;
steps++;
return desync ? -2 : 0;
}

/* Resumo barato do estado: se os dois lados divergirem, o hash muda */
uint8_t net_state_hash(void) {
    uint32_t h = (uint32_t)g.frame * 31 + g.score + g.state;
    for (int i = 0; i < MAX_PLAYERS; i++)
        h = h * 33 + g.players[i].pos.vx + g.players[i].pos.vz * 7 + g.players[i].hp;
    for (int i = 0; i < MAX_ENEMIES; i++)
        if (g.enemies[i].active) h = h * 33 + g.enemies[i].pos.vx + g.enemies[i].hp;
    return (uint8_t)(h ^ (h >> 8) ^ (h >> 16) ^ (h >> 24));
}

```

4. Integração no main.c. No topo, depois dos `#include`:**src/main.c**

```

#ifdef LINK_MODE
#include "net.h"
static int net_active = 0; /* 1 = jogando em lockstep */
static int net_request = 0; /* 1 = host, 2 = convidado (pedido na tela de título) */
#endif

```

Na tela de título, L1/R1 pedem a conexão (só registram o pedido; quem conecta é o laço principal):

src/main.c — game_tick()

```

case STATE_TITLE:
    effects_update();
#ifdef LINK_MODE
    if (g.in[0].pressed & PAD_L1) { net_request = 1; break; }
    if (g.in[0].pressed & PAD_R1) { net_request = 2; break; }
#endif
    who = any_pressed(PAD_START | PAD_CROSS);

```

No laço principal, troque o bloco de passos por este:

src/main.c — main()

```

input_update(g.in);
if (g.in[0].pressed & PAD_L2) /* L2 liga/desliga a depuração */
    show_debug ^= 1;

```

src/main.c — main() (cont.)

```

#ifdef LINK_MODE
    if (net_request) { /* conecta (trava até responder) */
        draw_center(120, net_request == 1 ? "AGUARDANDO O OUTRO JOGADOR..."
        : "PROCURANDO O HOST...");
    }

```

src/main.c — main() (cont.) (cont.)

```

    render_end_frame();
    uint32_t seed = (uint32_t)VSync(-1) * 2654435761u;
    net_init();
    if (net_handshake(net_request == 1, &seed) == 0) {
        srand(seed); /* mesma semente nos dois lados */
        sel[0].joined = sel[1].joined = 1;
        game_reset(); /* zera g (inclusive g.frame) */
        select_enter();
        net_active = 1;
    } else {
        show_message("SEM CONEXAO", 120);
    }
    net_request = 0;
    last_vsync = VSync(-1);
    continue;
}
if (net_active) {
    /* g.in vem da rede: os dois consoles veem as MESMAS entradas */
    INPUT local = g.in[0];
    for (int t = 0; t < ticks; t++) {
        int r = net_exchange(&local, g.in, net_state_hash());
        if (r == -1) {
            net_active = 0;
            show_message("CONEXAO PERDIDA", 180);
            break;
        }
        if (r == -2)
            show_message("DESSINCRONIZADO", 60);
        game_tick();
    }
} else
#endif
    for (int t = 0; t < ticks; t++) {
        game_tick();
        for (int i = 0; i < MAX_PLAYERS; i++)
            g.in[i].pressed = 0; /* "apertou agora" vale só 1 passo */
    }
}

```

Como o código funciona

- `net_exchange()` guarda a entrada local em `local_q[frame + NET_DELAY]`, envia, e — a partir do passo `NET_DELAY` — recebe **um** pacote: o que o outro lado enviou `NET_DELAY` passos atrás, que se refere exatamente ao passo atual. Os primeiros `NET_DELAY` passos usam entrada vazia nos dois lados.
- O jogador 1 é sempre o host e o 2 o convidado, nos dois consoles. `g.in[0]` e `g.in[1]` ficam idênticos nos dois lados.
- O `hash` enviado é o do passo atual do remetente; quem recebe compara com o hash que **ele** calculou `NET_DELAY` passos atrás (guardado em `hash_q`). Diferença = "DESSINCRONIZADO".
- O laço ainda roda 1 a 4 passos por quadro. Cada passo espera o pacote do outro, então os dois consoles avançam no ritmo do mais lento, mas a sequência de passos é a mesma.
- Sem resposta por 3 s, `net_exchange()` devolve `-1`: o jogo mostra "CONEXAO PERDIDA" e volta a ser local.

17.7 Testando com dois PCSX-Redux

1. Em `config.h`, troque `/* #define LINK_MODE */` por `#define LINK_MODE` e rode `./dev build`.
2. Abra **duas** janelas do PCSX-Redux carregando o mesmo `build/arena.cue` (ou o `.exe`).
3. Na janela **A**: *Configuration* → *Emulation*. Na janela *Emulation Configuration*, marque **Enable SIO1 Server** e confira **SIO1 Server Port** (padrão **6699**).
4. Na janela **B**: no mesmo lugar, preencha **SIO1 Client Host** com `127.0.0.1` e **SIO1 Client Port** com `6699`, e marque **Enable SIO1 Client**. O campo de host aceita só números e pontos: use o IP, não um nome.
5. Deixe **SIO1Mode** igual nas duas (o padrão, *Protobuf*, serve entre dois PCSX-Redux).
6. Na tela de título: aperte **L1** em A (host) e, em até ~30 s, **R1** em B (convidado). Os dois vão para a seleção com dois jogadores.
7. Escolha e jogue. Mova o jogador 1 em A e confira o mesmo movimento em B. Uma mensagem "DESSINCRONIZADO" indica um problema de determinismo (17.9).

DICA Se a conexão cair, o botão **Reconnect** (mesma janela) reconecta o cliente; reinicie a partida pelos dois lados.

17.8 Pela internet e em consoles reais

Dois PCs pela internet

- O lado **servidor** precisa receber conexões TCP na porta 6699. Opções: redirecionar a porta no roteador (*port forwarding*) para o IP local do PC, ou — mais simples e seguro — usar uma **VPN de jogo** (Tailscale, ZeroTier, Radmin VPN) e conectar no IP da VPN.
- Aumente `NET_DELAY` conforme o ping: com ping de 120 ms (ida e volta), a ida é ~60 ms → `NET_DELAY` 5-6.
- Os dois precisam do **mesmo** `.exe`. Distribua o build pronto, não o código.

Dois consoles com cabo link

- A porta serial existe nos modelos "grandes" (série SCPH-1000 a SCPH-9000) e foi **removida no PSone**.
- Grave o mesmo build em dois CDs (ou use um cartucho/ODE em cada console), ligue o cabo e use L1/R1 como acima.
- Console ↔ PC também é possível com um adaptador USB-serial de nível lógico compatível com a porta do PS1 (ela não usa níveis RS-232). Útil para depurar o protocolo com um script no PC.

17.9 Depurando dessincronias

1. Reproduza com `NET_DELAY` pequeno e os dois emuladores na mesma máquina.
2. Acrescente `printf("%d %02x\n", g.frame, net_state_hash());` antes de `game_tick()` no modo link. O texto aparece no log do PCSX-Redux (*Debug* → *Show Logs*). Compare os dois logs: o primeiro passo diferente aponta a hora do erro.
3. Torne o hash mais detalhado (inclua tiros, itens, `rand()` atual) para achar **o que** divergiu.
4. Suspeitos comuns: `rand()` chamado em `*_draw()`; variável `static` não zerada no reset; leitura do controle fora de `g.in`; `pressed` local; comportamento dependente de quantos quadros foram desenhados.

17.10 Limitações e próximos passos

Limitação	Caminho
Handshake trava a tela até conectar (até ~30 s)	fazer o handshake em etapas não bloqueantes, um pouco por quadro
Sem reconexão: caiu, volta ao modo local	guardar <code>g + frame</code> e reenviar o estado inteiro (<code>sizeof(GAME)</code> são poucos KB) ao reconectar
Dessincronia é só detectada	recuperar: o host envia <code>g</code> inteiro; o convidado substitui o seu
Atraso fixo	medir o tempo de ida e volta no handshake e escolher <code>NET_DELAY</code> automaticamente
2 jogadores	com <i>multitap</i> cada console pode ter 2 controles; o pacote passa a levar 2 entradas
Rollback (estilo GGPO)	possível em tese (todo o estado está em <code>g</code>), mas re-simular vários passos por quadro pesa no R3000

18. Desempenho e memória

A fase atual roda a ~30 FPS com folga para mais conteúdo. Ao crescer o jogo, meça antes de otimizar: aperte **L2** durante a partida para ver o overlay de depuração.

overlay L2

```
FPS 30 RAM GPU 61234/98304 <- bytes de primitivas no último quadro / PACKET_LEN
X 1408 Z 3712 ANG 1024 CAM 2048 <- posição e ângulo do jogador 1, ângulo da câmera
```

Sintoma	Causa provável	O que fazer
FPS cai perto de muitas paredes	polígonos demais na tela	reduza DRAW_DIST ; mapas com corredores (paredes escondem o resto)
Objetos somem ao acumular muita coisa	RAM GPU chegou ao PACKET_LEN	aumente PACKET_LEN (custa 2x em RAM) ou reduza polígonos
FPS cai com muitos inimigos	IA + colisão $O(n^2)$ + desenho	baixe MAX_ENEMIES ; modelos de inimigo com menos faces
Textura não cabe	VRAM cheia	sufixos -8bit/-4bit ; texturas 64x64
Jogo lento mas FPS ok	lógica pesada em game_tick (roda até 4x por quadro)	evite divisões e laços aninhados por passo

18.1 Orçamentos práticos

Recurso	Orçamento	Observação
Polígonos por quadro	~1 500–3 000 a 30 FPS	inclui chão e paredes visíveis
Faces por personagem	50–500	inimigos repetidos: prefira o lado baixo
Primitivas	96 KB por buffer (PACKET_LEN)	um quad texturizado usa ~40–52 bytes
Texturas	~320x480 px de 16 bits na área do empacotador	4 bits dá 4x mais
RAM	2 MB; o jogo usa ~450 KB	sons no SPU têm memória própria (512 KB)

18.2 Técnicas

- **Ponto fixo e deslocamentos** (>> 12) no lugar de divisões. Divisão no R3000 custa dezenas de ciclos.
- **Descarte cedo**: **render_mesh** descarta pelo raio do modelo antes de qualquer face; mantenha o **radius** correto (o exportador calcula).
- **Nível de detalhe**: modelo simplificado para inimigos distantes (escolha a malha pela distância à câmera).
- **Atualizar menos**: IA que "pensa" a cada 2 ou 4 passos (**(g.frame + i) & 3**), mantendo o movimento a cada passo.
- **Texturas de 4/8 bits** com paletas: menos VRAM e o mesmo custo de desenho.

19. Depuração

19.1 Ferramentas

Ferramenta	Como usar
<code>printf</code>	vai para o TTY da BIOS. No PCSX-Redux: <i>Debug</i> → <i>Show Logs</i> . Ótimo para seguir valores por passo
Depurador do PCSX-Redux	<i>Configuration</i> → <i>Emulation</i> → <i>Enable Debugger</i> : pontos de parada, memória, VRAM, registradores
Servidor GDB	<i>Enable GDB Server</i> (porta padrão 3333). Conecte com <code>gdb-multiarch build/arena.elf</code> e <code>target remote localhost:3333</code>
Visualizador de VRAM	PCSX-Redux mostra a VRAM ao vivo; compare com <code>./dev vram</code>
<code>build/arena.map</code>	endereço e tamanho de cada símbolo; útil para achar quem ocupa memória ou onde parou
Overlay L2	FPS, bytes de primitivas, posição — dentro do jogo
Capturas de tela	a função de captura do emulador, para comparar antes/depois de uma mudança

19.2 Falhas comuns

Sintoma	Causa provável	Correção
Tela preta logo ao iniciar, depois de adicionar código	função da <code>libgcc</code> com ABI incompatível (ex.: <code>__builtin_bswap32</code>), ou ponteiro nulo	troque o builtin por código próprio (ver R8); confira <code>VERBOSE=1 ./dev build</code> por avisos "linking abicalls" de símbolos novos
Jogo congela num ponto	laço esperando algo que não vem (CD, SPU, serial)	use versões com limite de tempo (como <code>read_byte_timeout</code> da R11); <code>printf</code> antes e depois
Modelo some ou aparece do avesso	normais invertidas / ordem dos vértices	Blender: <i>Recalculate Outside</i> ; ou <code>DRAW_NOCULL</code> para testar
Objeto pisca atrás do chão	ordem na OT	ajuste <code>zbias</code> (positivo = mais ao fundo)
Textura com lixo	VRAM sobreposta (TIM manual em <code>assets/tim/</code>)	<code>./dev vram</code> ; mova o TIM
Comportamento diferente entre emuladores	memória não inicializada, temporização	inicialize tudo; teste no DuckStation e no PCSX-Redux
Valores "malucos" após muito tempo	estouro de inteiro	revise multiplicações (capítulo 4.1)
Aviso <code>abicalls</code> / <code>-mhard-float</code> no link	normal neste toolchain para símbolos da SDK	só preocupa se um símbolo novo da <code>libgcc</code> aparecer

DICA Para ver quais rotinas da `libgcc` entraram no executável: `./dev shell` e `mipsel-linux-gnu-nm build/arena.elf | grep -E ' __[a-z]+[sd]i[23]$'`. O ideal é não aparecer nada.

20. Fluxo de trabalho, versões e entrega

20.1 Git

- Uma *branch* por recurso (*feature/inimigo-atirador*), *commits* pequenos com o porquê na mensagem.
- Versione os *.blend* e os *models/*.h* gerados — o projeto compila sem o Blender.
- Não versione *build/*, *.sdk/*, *dev.conf*, *CLAUDE.local.md* (já no *.gitignore*).
- O *.gitattributes* força LF nos scripts; não o remova.
- Builds prontos (*arena.cue/.bin*) vão em **Releases** do GitHub, não no repositório.

20.2 Ao terminar uma mudança

1. *./dev build* sem erros nem avisos novos.
2. Teste no emulador o recurso novo e a lista de verificação abaixo.
3. Atualize *docs/GUIA.md* (receitas) e/ou *README.md* (técnico) se outra pessoa precisa saber.
4. Uma linha em *docs/HISTORICO.md* → *Registro de mudanças* (o agente de IA lê esse arquivo).

20.3 Lista de verificação de release

- *./dev clean && ./dev build* sem erros nem avisos novos.
- Título → seleção → partida com 1 jogador; com 2 controles, personagens diferentes e respawn.
- Vitória e game over; START reinicia, SELECT volta à seleção (com a R3: passa de fase).
- Overlay L2: FPS estável e *RAM GPU* abaixo do limite no pior ponto da fase.
- *arena.cue* inicia no DuckStation e no PCSX-Redux.
- Se mexeu em lógica: um teste com *LINK_MODE* (se você usa o modo link).

20.4 Gravar em CD

Use *build/arena.cue* + *arena.bin* num gravador de CD-R (ImgBurn no Windows, modo *Disc-At-Once*, velocidade baixa). O console precisa de desbloqueio (modchip ou exploit de boot) para rodar CD-R. Consoles japoneses e alguns PAL exigem a licença da Sony no setor de boot — veja o comentário no *iso.xml*. Detalhes no *docs/GUIA.md*, seção 13.

20.5 Trabalhando com um agente de IA

O *CLAUDE.md* na raiz descreve as regras do projeto para o agente (ambiente, comandos, regras de ouro, armadilhas). Peça mudanças citando a receita ("implemente a R1 do manual") e sempre exija *./dev build* no fim. O agente não vê o emulador: descreva o que testou.

21. Solução de problemas

Problema	Solução
Docker não está respondendo	abra o Docker Desktop (<i>WSL Integration</i> ligado) ou <code>sudo service docker start</code>
permission denied ... docker.sock	<code>sudo usermod -aG docker \$USER</code> e reabra o terminal
/usr/bin/env: 'bash\r'	finais de linha CRLF: <code>sed -i 's/\r\$//' dev</code>
build muito lento	projeto em <code>/mnt/c</code> ; mova para <code>~/</code> no disco do Linux
Emulador não configurado	preencha <code>EMULATOR</code> no <code>dev.conf</code> (<code>./dev doctor</code> confere)
modelo novo "não existe"	o símbolo vem do nome do arquivo: <code>models/heroi.h</code> → <code>heroi_mesh</code>
Vértice fora do limite de 16 bits	modelo grande demais para a escala; reduza ou use <code>psx_scale</code>
VRAM cheia	texturas demais/grandes; sufixos <code>-8bit/-4bit</code>
erro de <code>struct</code> com "too many initializers"	linha da tabela com campos a mais que o tipo; confira a ordem dos campos
"excess elements" / campos trocados após editar um tipo	campo novo no meio da <code>struct</code> : mova-o para o fim
função nova "undefined reference"	protótipo em <code>game.h</code> sem a função em nenhum <code>.c</code> , ou nome diferente
<code>implicit declaration of function</code>	falta o protótipo em <code>game.h</code> (ou o <code>#include</code>)
mudança em <code>models/</code> ou <code>assets/</code> não aparece	<code>./dev clean && ./dev build</code>
modo link não conecta	mesma versão do <code>.exe</code> ; IP numérico no <i>SIO1 Client Host</i> ; porta 6699 liberada; L1 primeiro no host

Apêndice A — Caracteres do mapa

Caractere	Significado	Onde é tratado
#	parede	level_load (sólido) e build_geometry
.	chão	—
	vazio (sólido, sem chão)	—
P	início dos jogadores	level_load
C	caixa destrutível	crate_spawn
G H N W	gema, vida, energia, arma nova	pickup_spawn
V	velocidade (R4)	pickup_spawn(PICK_SPEED...)
E B F	GRUNT, BRUTO, VELOZ	enemy_defs[].map_char
S K	ATIRADOR (R1), REI SOMBRIO (R2)	enemy_defs[].map_char
1...9	objetos de cenário	prop_defs[].map_char

Mapa da fase 1, para referência:

src/level.c — Arena

```

"#####",
"#P.....#.....C.....#",
"#.....#..E.....G..#",
"#..CC...#.....####...#",
"#.....H#..#..E..#",
"#...G.....#W.#...#",
"#####...####...##...#",
"#.....#..#..1.....1..#",
"#..E...#G.#.....F.....#",
"#.....#..#.....C.C...#",
"#..N.....E.....#",
"#...####...####...#",
"#...#..#..B...#G.#...#",
"#.G.#..#.....#..E..#",
"#...##..#..C.C...##...#",
"#.F.....W.....#",
"#...E...H.....B...#",
"#..C.....G...F...N..#",
"#..1.....2.....2...1..#",
"#####",

```

Apêndice B — Constantes de config.h

Constante	Valor	Efeito
SCREEN_W	320	
SCREEN_H	240	
CLEAR_R	20	cor do céu/fundo
CLEAR_G	24	
CLEAR_B	48	
OT_LEN	2048	níveis de profundidade da Ordering Table
PACKET_LEN	(96*1024)	memória para primitivas por quadro
NEAR_Z	40	polígonos mais perto que isso são descartados
DRAW_DIST	4200	objetos além disso não são desenhados
FOV_H	160	"distância da tela": maior = zoom maior
TILE_SIZE	256	tamanho de um bloco do mapa
WALL_HEIGHT	384	altura das paredes
GRAVITY	3	aceleração para baixo por quadro
CAM_DIST	820	distância atrás do jogador
CAM_MIN_DIST	260	o mais perto que chega ao encostar na parede
CAM_HEIGHT	600	altura acima do jogador
CAM_LOOK_UP	120	ponto que a câmera mira (acima do pé)
CAM_TURN_SPEED	40	giro com L1/R1 por quadro
PLAYER_MAX_ENERGY	100	
PLAYER_RADIUS	70	
PLAYER_INVULN	60	quadros invulnerável após levar dano
ENERGY_REGEN_DELAY	20	a cada N quadros recupera 1 de energia
MAX_PLAYERS	2	jogadores ao mesmo tempo (porta 1 e 2)
RESPAWN_TIME	300	5 s caído antes de voltar (com 2 jogadores)
MAX_ENEMIES	24	
MAX_BULLETS	48	
MAX_PICKUPS	24	
MAX_CRATES	32	
MAX_EFFECTS	16	
MAX_PROPS	64	

Apêndice C — Botões (psxpad.h)

Constante	Bit	Uso no jogo
PAD_SELECT	0	trocar skin; nas telas de fim, voltar à seleção
PAD_START	3	começar, pausar, entrar como jogador 2
PAD_UP/RIGHT/DOWN/LEFT	4-7	andar (relativo à câmera)
PAD_L2	8	overlay de depuração
PAD_R2	9	trocar poder
PAD_L1 / PAD_R1	10 / 11	girar a câmera; no título com LINK_MODE: host / convidado
PAD_TRIANGLE	12	trocar arma
PAD_CIRCLE	13	usar poder; na seleção, voltar
PAD_CROSS	14	pular; confirmar
PAD_SQUARE	15	atirar (segurar)

Teste: `if (in->held & PAD_SQUARE)` (segurando) ou `if (in->pressed & PAD_CROSS)` (acabou de apertar).

Apêndice D — Glossário

Termo	Definição
CLUT	<i>Color Look-Up Table</i> : paleta de uma textura de 4 ou 8 bits, guardada na VRAM
Determinismo	mesmas entradas + mesmo estado inicial → mesmo resultado. Base do lockstep
GTE	<i>Geometry Transformation Engine</i> : coprocessador que gira, projeta e ilumina vértices
Lockstep	modo multijogador em que os lados trocam só entradas e simulam o mesmo jogo
OT	<i>Ordering Table</i> : listas de primitivas por profundidade, desenhadas do fundo para a frente
Passo (tick)	1/60 s de lógica; unidade de tempo de todo o jogo
Ponto fixo	inteiro escalado (ONE = 4096 = 1,0) no lugar de float
Pool	vetor estático de entidades com flag active
SIO1	porta serial do PS1 (cabo link); psxsio no PSn00bSDK
SPU	processador de som: 24 vozes ADPCM, 512 KB
TIM	formato de imagem do PS1, com posição na VRAM
tpage	página de textura: região de 64×256 (16 bits) da VRAM + modo de cor
VAG	formato de amostra de som ADPCM do PS1
VSync	retraço vertical (60 Hz NTSC); o jogo sincroniza o desenho com ele
zbias	deslocamento na OT para corrigir a ordem de desenho de um modelo

Apêndice E — Roteiro de exercícios

Uma sequência para aprender o código mexendo, do mais simples ao mais profundo:

#	Exercício	Aprende
1	Mude <code>CAM_DIST</code> , <code>GRAVITY</code> e o pulo do ROBO	ciclo editar-compilar-testar
2	Desenhe uma fase nova no mapa de texto	<code>level_load</code> , caracteres
3	Crie uma arma e um inimigo só com tabelas	orientação a dados
4	Modele um personagem no Blender e coloque na seleção	exportador, <code>character_defs</code>
5	R4 (item de velocidade)	fluxo de um recurso por vários arquivos
6	R1 + R2 (atirador e chefe)	pools compartilhados, HUD
7	R3 + R12 (fases e introdução)	máquina de estados, progressão
8	R6 e R5 (congelar, explosão)	estado por entidade, dano em área
9	R8 (som)	SPU, CMake, conversão de assets
10	R11 (modo link) entre dois PCSX-Redux	determinismo, protocolo, depuração distribuída